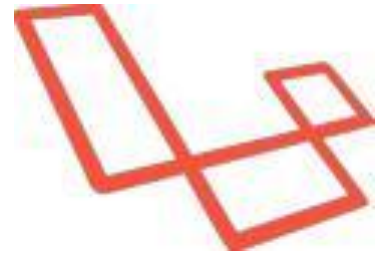


Chapter – 2.1 [Introduction to Laravel]

☺ What is Laravel?



Laravel is a free, open-source PHP web framework, created by Taylor Otwell and intended for the development of web applications following the model–view–controller (MVC) architectural pattern. Some of the features of Laravel are a modular packaging system with a dedicated dependency manager, different ways for accessing relational databases, utilities that aid in application deployment and maintenance, and its orientation toward syntactic sugar.

As of March 2015, Laravel is regarded as one of the most popular PHP frameworks, together with Symfony2, Nette, CodeIgniter, Yii2 and other frameworks.

The source code of Laravel is hosted on GitHub and licensed under the terms of MIT License

☺ Features:

1. Automatic pagination

The paginate method provided by Laravel automatically takes care of setting the proper limit and offset based on the current page being viewed by the user.

2. View composers

The view composer is one of the most important features of Laravel which makes sure that the controllers load a bunch of data from models for views which is not related to that method's page content.

3. Routing system

Laravel provides the flexibility to its developers to choose which route is triggered on the application. Laravel routes are defined in the app/Http/routes.php file, which is automatically loaded by the Laravel framework. The most basic Laravel routes simply accept a URI and a Closure, providing a very simple and expressive method of defining routes. The Laravel developer just need to enter the name of the route and it will automatically insert the correct URI.

4. Unit testing

Laravel enables its web artisans to test their code in the best way possible. Using the Artisan command line utility, developers can write their own multiple unit tests easily.

☺ MVC architecture:

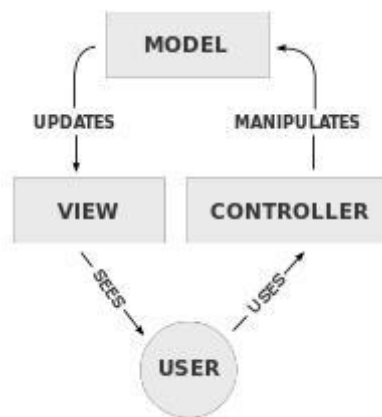
Model–view–controller (MVC) is a software architectural pattern for implementing user interfaces on computers. It divides a given software application into three interconnected parts, so as to separate internal representations of information from the ways that information is presented to or accepted from the user

Traditionally used for desktop graphical user interfaces (GUIs), this architecture has become extremely popular for designing web applications.

DESCRIPTION

As with other software patterns, MVC expresses the "core of the solution" to a problem while allowing it to be adapted for each system. Particular MVC architectures can vary significantly from the traditional description here.

COMPONENTS



A typical collaboration of the MVC components.

The central component of MVC, the *model*, captures the behavior of the application in terms of its problem domain, independent of the user interface.

- The model directly manages the data, logic and rules of the application.
- A *view* can be any output representation of information, such as a chart or a diagram. Multiple views of the same information are possible, such as a bar chart for management and a tabular view for accountants.
- The third part, the *controller*, accepts input and converts it to commands for the model or view.

INTERACTIONS

In addition to dividing the application into three kinds of components, the model–view– controller design defines the interactions between them.

- A *model* stores data that is retrieved according to commands from the controller and displayed in the view.
- A *view* generates new output to the user based on changes in the model.
- A *controller* can send commands to the model to update the model's state (e.g. editing a document). It can also send commands to its associated view to change the view's presentation of the model (e.g. by scrolling through a document).

☺ Structure of laravel application (laravel directory structure):

INTRODUCTION

The default Laravel application structure is intended to provide a great starting point for both large and small applications. Of course, you are free to organize your application however you like. Laravel imposes almost no restrictions on where any given class is located - as long as Composer can autoload the class.

THE ROOT DIRECTORY

The root directory of a fresh Laravel installation contains a variety of directories:

The `app` directory, as you might expect, contains the core code of your application. We'll explore this directory in more detail soon.

The `bootstrap` directory contains a few files that bootstrap the framework and configure autoloading, as well as a `cache` directory that contains a few framework generated files for bootstrap performance optimization.

The `config` directory, as the name implies, contains all of your application's configuration files.

The `database` directory contains your database migration and seeds. If you wish, you may also use this directory to hold an SQLite database.

The `public` directory contains the front controller and your assets (images, JavaScript, CSS, etc.).

The `resources` directory contains your views, raw assets (LESS, SASS, CoffeeScript), and localization files.

The `storage` directory contains compiled Blade templates, file based sessions, file caches, and other files generated by the framework. This directory is segregated into `app`, `framework`, and `logs` directories. The `app` directory may be used to store any files utilized by your application. The `framework` directory is used to store framework generated files and caches. Finally, the `logs` directory contains your application's log files.

The `tests` directory contains your automated tests. An example [PHPUnit](#) is provided out of the box.

The `vendor` directory contains your [Composer](#) dependencies.

THE APP DIRECTORY

The "meat" of your application lives in the `app` directory. By default, this directory is namespaced under `App` and is autoloaded by Composer using the [PSR-4 autoloading standard](#).

The `app` directory ships with a variety of additional directories such as `Console`, `Http`, and `Providers`. Think of the `Console` and `Http` directories as providing an API into the "core" of your application. The HTTP protocol and CLI are both mechanisms to interact with your application, but do not actually contain application logic. In other words, they are simply two ways of issuing commands to your application. The `Console` directory contains all of your Artisan commands, while the `Http` directory contains your controllers, middleware, and requests.

The `Events` directory, as you might expect, houses [event classes](#). Events may be used to alert other parts of your application that a given action has occurred, providing a great deal of flexibility and decoupling.

The `Exceptions` directory contains your application's exception handler and is also a good place to stick any exceptions thrown by your application.

The `Jobs` directory, of course, houses the [queueable jobs](#) for your application. Jobs may be queued by your application or run synchronously within the current request lifecycle.

The `Listeners` directory contains the handler classes for your events. Handlers receive an event and perform logic in response to the event being fired. For example, a `UserRegistered` event might be handled by a `SendWelcomeEmail` listener.

The `Policies` directory contains the authorization policy classes for your application. Policies are used to determine if a user can perform a given action against a resource. For more information, check out the [authorization documentation](#).

Note: Many of the classes in the `app` directory can be generated by Artisan via commands. To review the available commands, run the `php artisan list make` command in your terminal.

Chapter – 2.2 [Installation]

☺ Basic requirements for Laravel:

SERVER REQUIREMENTS

The Laravel framework has a few system requirements. Of course, all of these requirements are satisfied by the Laravel Homestead virtual machine, so it's highly recommended that you use Homestead as your local Laravel development environment.

However, if you are not using Homestead, you will need to make sure your server meets the following requirements:

- PHP >= 5.6.4
- OpenSSL PHP Extension
- PDO PHP Extension
- Mbstring PHP Extension
- Tokenizer PHP Extension

☺ Using Laravel Installer:

First, download the Laravel installer using Composer:

```
composer global require "laravel/installer"
```

Make sure to place the `~/.composer/vendor/bin` directory (or the equivalent directory for your OS) in your `$PATH` so the `laravel` executable can be located by your system.

Once installed, the `laravel new` command will create a fresh Laravel installation in the directory you specify. For instance, `laravel new blog` will create a directory named `blog` containing a fresh Laravel installation with all of Laravel's dependencies already installed:

```
laravel new blog
```

☺ Using Composer:

Alternatively, you may also install Laravel by issuing the Composer `create-project` command in your terminal:

```
composer create-project laravel/laravel your-project-name 5.2.*
composer create-project --prefer-dist laravel/laravel projectname
For latest version [5.4]
    composer create-project laravel/laravel your-project-name
    composer create-project laravel/laravel projectname --prefer-dist
```

☺ How does Composer work?

Composer is a tool for dependency management in PHP. It allows you to declare the libraries your project depends on and it will manage (install/update) them for you.

DEPENDENCY MANAGEMENT

Composer is **not** a package manager in the same sense as Yum or Apt are. Yes, it deals with "packages" or libraries, but it manages them on a per-project basis, installing them in a directory (e.g. `vendor`) inside your project. By default it does not install anything globally. Thus, it is a dependency manager. It does however support a "global" project for convenience via the `global` command.

This idea is not new and Composer is strongly inspired by node's `npm` and ruby's `bundler`.

Suppose:

1. You have a project that depends on a number of libraries.
2. Some of those libraries depend on other libraries.

Composer:

1. Enables you to declare the libraries you depend on.
2. Finds out which versions of which packages can and need to be installed, and installs them (meaning it downloads them into your project).

See the Basic usage chapter for more details on declaring dependencies.

SYSTEM REQUIREMENTS

Composer requires PHP 5.3.2+ to run. A few sensitive php settings and compile flags are also required, but when using the installer you will be warned about any incompatibilities.

To install packages from sources instead of simple zip archives, you will need git, svn, fossil or hg depending on how the package is version-controlled.

Composer is multi-platform and we strive to make it run equally well on Windows, Linux and OSX.

☺ Installation – Linux & Windows:

Installation – Linux / Unix / OSX

DOWNLOADING THE COMPOSER EXECUTABLE

Composer offers a convenient installer that you can execute directly from the commandline. Feel free to [download this file](#) or review it on [GitHub](#) if you wish to know more about the inner workings of the installer. The source is plain PHP.

There are in short, two ways to install Composer. Locally as part of your project, or globally as a system wide executable.

Locally

Installing Composer locally is a matter of just running the installer in your project directory. See [the Download page](#) for instructions.

The installer will just check a few PHP settings and then download `composer.phar` to your working directory. This file is the Composer binary. It is a PHAR (PHP archive), which is an archive format for PHP which can be run on the command line, amongst other things.

Now just run `php composer.phar` in order to run Composer.

You can install Composer to a specific directory by using the `--install-dir` option and additionally (re)name it as well using the `--filename` option. When running the installer when following [the Download page instructions](#) add the following parameters:

```
php composer-setup.php --install-dir=bin --filename=composer
```

Now just run `php bin/composer` in order to run Composer.

Globally

You can place the Composer PHAR anywhere you wish. If you put it in a directory that is part of your `PATH`, you can access it globally. On unixy systems you can even make it executable and invoke it without directly using the `php` interpreter.

After running the installer following [the Download page instructions](#) you can run this to move `composer.phar` to a directory that is in your path:

```
mv composer.phar /usr/local/bin/composer
```

Note: If the above fails due to permissions, you may need to run it again with `sudo`.

Note: On some versions of OSX the `/usr` directory does not exist by default. If you receive the error `"/usr/local/bin/composer: No such file or directory"` then you must create the directory manually before proceeding: `mkdir -p /usr/local/bin`.

Note: For information on changing your `PATH`, please read the [Wikipedia article](#) and/or use Google.

Now just run `composer` in order to run Composer instead of `php composer.phar`.

INSTALLATION - WINDOWS

USING THE INSTALLER

This is the easiest way to get Composer set up on your machine.

Download and run Composer-Setup.exe. It will install the latest Composer version and set up your PATH so that you can just call `composer` from any directory in your command line.

Note: Close your current terminal. Test usage with a new terminal: This is important since the PATH only gets loaded when the terminal starts.

MANUAL INSTALLATION

Change to a directory on your PATH and run the installer following the Download page instructions to download `composer.phar`.

Create a new `composer.bat` file alongside `composer.phar`:

```
C:\bin>echo @php "%~dp0composer.phar" %*>composer.bat
```

Add the directory to your PATH environment variable if it isn't already. For information on changing your PATH variable, please see this article and/or use Google.

Close your current terminal. Test usage with a new terminal:

```
C:\Users\username>composer -V
Composer version 1.0.0 2016-01-10 20:34:53
```

☺ Finding and Installing new packages:

Using the search on <http://packagist.org>, you can find packages to add common features, such as image manipulation or PDF generation to your application. Indicators of good packages beyond the number of downloads and the number of stars on GitHub are the quality of documentation, the test coverage, and the overall project activity. Before adding a new package, you can also browse the different versions of a package and its dependencies on Packagist:

laravel/framework

composer require laravel/framework

PackageistThe PHP Package Repository

BrowseSubmitCreate accountSign in

HomePageSourceIssues

Installs: 6 195 548

Dependencies: 331

Stars: 4 068

Watchers: 652

Forks: 2 315

Open Issues: 76

Language: PHP

v5.1.16

2019-09-04 13:46 UTC

dev-master (v5.2.x-dev)

v5.1.16

v5.1.15

v5.1.14

v5.1.13

v5.1.12

v5.1.11

v5.1.10

v5.1.8

v5.1.6

v5.1.4

requires

requires (dev)

suggests

provides

conflicts

replaces

MIT

e34d3c37ef560248ae5728161c256ff6b4e1

Taylor Otwell<@taylorotwell@gmail.com>

#laravel

README

Readme

Downloads: 9.2 M

Installs: 6 195 548

Uninstall: 0.2 M

License: MIT

Note: This repository contains the core code of the Laravel framework. If you want to build an application using Laravel 5, visit the main [Laravel repository](#).

Laravel PHP Framework

Laravel is a web application framework with expressive, elegant syntax. We believe development must be an enjoyable, creative experience to be truly fulfilling. Laravel attempts to take the pain out of development by easing common tasks used in the majority of web projects, such as authentication, routing, sessions, queuing, and caching.

Laravel is accessible, yet powerful, providing powerful tools needed for large, robust applications. A superb inversion of control container, expressive migration system, and tightly integrated unit testing support give you the tools you need to build any application with which you are tasked.

[Official Documentation](#)

About Packageist

Statistics

Atom/RSS Feeds

Browse Packages

Packageist maintenance and hosting is supported by [Toran Proxy](#)



In development it is fine to use a `dev-master` branch, but in production, it is better to stick with a precise version number to avoid potential compatibility issues.

To install a package, open `composer.json` in a text editor and insert its name and the desired version in the `require` object:

```
"require": { "laravel/laravel":  
    "v5.1.*",  
    "intervention/image": "dev-master"  
}
```

Since it is a JSON file, you need to be careful not to leave any trailing commas on the last line. To check if there are any errors in your `composer.json` file, you can use the `composer validate` command.

Then, simply run `composer install` and Composer will fetch the package and its dependencies. To update the `composer.lock` file and save the exact version numbers of the resolved dependencies, call `composer update`. If you deploy or distribute your application, the lock file allows everyone else to retrieve the exact same packages when they run `composer install`. The `update` command, on the other hand, will always check for the latest version of every package and if you run it in production, you risk running into compatibility issues.

Chapter – 2.3 [Configuration]

☺ Introduction:

INTRODUCTION

All of the configuration files for the Laravel framework are stored in the `config` directory. Each option is documented, so feel free to look through the files and get familiar with the options available to you.

ACCESSING CONFIGURATION VALUES

You may easily access your configuration values using the global `config` helper function from anywhere in your application. The configuration values may be accessed using "dot" syntax, which includes the name of the file and option you wish to access. A default value may also be specified and will be returned if the configuration option does not exist:

```
$value = config('app.timezone');
```

To set configuration values at runtime, pass an array to the `config` helper:

```
config(['app.timezone' => 'America/Chicago']);
```

☺ Environment configuration:

It is often helpful to have different configuration values based on the environment the application is running in. For example, you may wish to use a different cache driver locally than you do on your production server.

To make this a cinch, Laravel utilizes the DotEnv PHP library by Vance Lucas. In a fresh Laravel installation, the root directory of your application will contain a `.env.example` file. If you install Laravel via Composer, this file will automatically be renamed to `.env`. Otherwise, you should rename the file manually.

Retrieving Environment Configuration

All of the variables listed in this file will be loaded into the `$_ENV` PHP super-global when your application receives a request. However, you may use the `env` helper to retrieve values from these variables in your configuration files. In fact, if you review the Laravel configuration files, you will notice several of the options already using this helper:

```
'debug' => env('APP_DEBUG', false),  
var_dump(getenv('APP_KEY'));  
    echo "<br>";  
var_dump(getenv('APP_LOG'));
```

The second value passed to the `env` function is the "default value". This value will be used if no environment variable exists for the given key.

Your `.env` file should not be committed to your application's source control, since each developer / server using your application could require a different environment configuration.

If you are developing with a team, you may wish to continue including a `.env.example` file with your application. By putting place-holder values in the example configuration file, other developers on your team can clearly see which environment variables are needed to run your application.

DETERMINING THE CURRENT ENVIRONMENT

The current application environment is determined via the `APP_ENV` variable from your `.env` file. You may access this value via the `environment` method on the App facade:

```
$environment = App::environment();
```

You may also pass arguments to the `environment` method to check if the environment matches a given value. The method will return `true` if the environment matches any of the given values:

```
if (App::environment('local')) {  
    // The environment is local  
}  
  
if (App::environment('local', 'staging')) {  
    // The environment is either local OR staging...  
}
```

An application instance may also be accessed via the `app` helper method:

```
$environment = app()->environment();
```

CONFIGURATION CACHING

To give your application a speed boost, you should cache all of your configuration files into a single file using the `config:cache` Artisan command. This will combine all of the configuration options for your application into a single file which will be loaded quickly by the framework.

You should typically run the `php artisan config:cache` command as part of your production deployment routine. The command should not be run during local development as configuration options will frequently need to be changed during the course of your application's development.

☹ Protecting sensitive configuration

It is advisable to keep all of your sensitive configuration out of your configuration files. Things such as database passwords, Stripe API keys, and encryption keys should be kept out of your configuration files whenever possible. So, where should we place them? Thankfully, Laravel provides a very simple solution to protecting these types of configuration items using "dot" files.

Next, create a `.env.local.php` file within the root of your project, which is usually the same directory that contains your `composer.json` file. The `.env.local.php`

```
<?php
```

```
return array(  
    'TEST_STRIPE_KEY' => 'super-secret-sauce',  
);
```

MAINTENANCE MODE

When your application is in maintenance mode, a custom view will be displayed for all requests into your application. This makes it easy to "disable" your application while it is updating or when you are performing maintenance. A maintenance mode check is included in the default middleware stack for your application. If the application is in maintenance mode, a `MaintenanceModeException` will be thrown with a status code of 503.

To enable maintenance mode, simply execute the `down` Artisan command:

```
php artisan down
```

You may also provide `message` and `retry` options to the `down` command. The `message` value may be used to display or log a custom message, while the `retry` value will be set as the HTTP header's value:

```
php artisan down --message='Upgrading Database' --retry=60
```

To disable maintenance mode, use the `up` command:

```
php artisan up
```

Maintenance Mode Response Template

The default template for maintenance mode responses is located in `resources/views/errors/503.blade.php`. You are free to modify this view as needed for your application.

Maintenance Mode & Queues

While your application is in maintenance mode, no queued jobs will be handled. The jobs will continue to be handled as normal once the application is out of maintenance mode.

Alternatives To Maintenance Mode

Since maintenance mode requires your application to have several seconds of downtime, you may consider alternatives like Envoyer to accomplish zero-downtime deployment with Laravel.

☺ Database Configuration:

Laravel makes connecting with databases and running queries extremely simple across a variety of database back-ends using either raw SQL, the fluent query builder, and the Eloquent ORM. Currently, Laravel supports four database systems:

- MySQL
- Postgres
- SQLite
- SQL Server

CONFIGURATION

Laravel makes connecting with databases and running queries extremely simple. The database configuration for your application is located at `config/database.php`. In this file you may define all of your database connections, as well as specify which connection should be used by default. Examples for all of the supported database systems are provided in this file.

By default, Laravel's sample environment configuration is ready to use with Laravel Homestead, which is a convenient virtual machine for doing Laravel development on your local machine. Of course, you are free to modify this configuration as needed for your local database.

SQLite Configuration

After creating a new SQLite database using a command such as `touch database/database.sqlite`, you can easily configure your environment variables to point to this newly created database by using the database's absolute path:

```
DB_CONNECTION=sqlite
DB_DATABASE=/absolute/path/to/database.sqlite
```

SQL Server Configuration

Laravel supports SQL Server out of the box; however, you will need to add the connection configuration for the database:

```
'sqlsrv' => [
    'driver' => 'sqlsrv',
    'host' => env('DB_HOST', 'localhost'),
    'database' => env('DB_DATABASE', 'forge'),
    'username' => env('DB_USERNAME', 'forge'),
    'password' => env('DB_PASSWORD', ''),
    'charset' => 'utf8',
    'prefix' => '',
],
```

Read / Write Connections

Sometimes you may wish to use one database connection for SELECT statements, and another for INSERT, UPDATE, and DELETE statements. Laravel makes this a breeze, and the proper connections will always be used whether you are using raw queries, the query builder, or the Eloquent ORM.

To see how read / write connections should be configured, let's look at this example:

```
'mysql' => [ 'read'
    => [
        'host' => '192.168.1.1',
    ],
    'write' => [
        'host' => '196.168.1.2'
    ],
    'driver'     => 'mysql',
    'database'   => 'database',
    'username'   => 'root',
    'password'   => '',
    'charset'    => 'utf8',
    'collation'  => 'utf8_unicode_ci',
    'prefix'     => '',
],
```

Note that two keys have been added to the configuration array: `read` and `write`. Both of these keys have array values containing a single key: `host`. The rest of the database options for the `read` and `write` connections will be merged from the main `mysql` array.

So, we only need to place items in the `read` and `write` arrays if we wish to override the values in the main array. So, in this case, `192.168.1.1` will be used as the "read" connection, while `192.168.1.2` will be used as the "write" connection. The database credentials, prefix, character set, and all other options in the main `mysql` array will be shared across both connections.

Chapter – 3.1 [Artisan]

☺ Artisan command line tool:

ARTISAN CONSOLE

Introduction

Artisan is the name of the command-line interface included with Laravel. It provides a number of helpful commands for your use while developing your application. It is driven by the powerful Symfony Console component. To view a list of all available Artisan commands, you may use the `list` command:

```
php artisan list
```

Every command also includes a "help" screen which displays and describes the command's available arguments and options. To view a help screen, simply precede the name of the command with `help`:

```
php artisan help migrate
```

Writing Commands

In addition to the commands provided with Artisan, you may also build your own custom commands for working with your application. You may store your custom commands in the `app/Console/Commands` directory; however, you are free to choose your own storage location as long as your commands can be autoloaded based on your `composer.json` settings.

To create a new command, you may use the `make:console` Artisan command, which will generate a command stub to help you get started:

```
php artisan make:console commandname
```

```
php artisan make:console SendEmails
```

For new version:

```
php artisan make:command commandname
```

The command above would generate a class at `app/Console/Commands/SendEmails.php`. When creating the command, the option may be used to assign the terminal

command name:

```
php artisan make:console SendEmails --command=emails:send
```

Command Structure

Once your command is generated, you should fill out the `signature` and `description` properties of the class, which will be used when displaying your command on the `list` screen.

The `handle` method will be called when your command is executed. You may place any command logic in this method. Let's take a look at an example command.

Note that we are able to inject any dependencies we need into the command's constructor. The Laravel service container will automatically inject all dependencies type-hinted in the constructor. For greater code reusability, it is good practice to keep your console commands light and let them defer to application services to accomplish their tasks.

```
<?php

namespace App\Console\Commands;

use Illuminate\Console\Command;
use Mail;

class Email extends Command
{
    /**
     * The name and signature of the console command.
     *
     * @var string
     */
    protected $signature = 'class:email';

    /**
     * The console command description.
     *
     * @var string
     */
    protected $description = 'Command description';

    /**
     * Create a new command instance.
     *
     * @return void
     */
    public function __construct()
    {
        parent::__construct();
    }

    /**
     * Execute the console command.
     *
     * @return mixed
     */
    public function handle()
    {
        $user = ['foo' => 'bar'];
        Mail::send('emails.mailExample', ['user' => $user], function ($m) use ($user) {
            $m->from('vipulagravat5@gmail.com', 'Your Application');

            $m->to('mnlbhanshali@gmail.com', 'Student')->subject('Your Reminder!');
        });
        echo "Send Mail successfully";
    }
}
```

COMMAND I/O

Defining Input Expectations

When writing console commands, it is common to gather input from the user through arguments or options. Laravel makes it very convenient to define the input you expect from the user using the `signature` property on your commands. The `signature` property allows you to define the name, arguments, and options for the command in a single, expressive, route-like syntax.

All user supplied arguments and options are wrapped in curly braces. In the following example, the command defines one **required** argument: `user`:

```
/**
 * The name and signature of the console command.
 *
 * @var string
 */
protected $signature = 'email:send {user}';
```

You may also make arguments optional and define default values for optional arguments:

```
// Optional argument...
email:send {user?}

// Optional argument with default value...
email:send {user=foo}
```

Options, like arguments, are also a form of user input. However, they are prefixed by two hyphens (`--`) when they are specified on the command line. We can define options in the signature like so:

```
/**
 * The name and signature of the console command.
 *
 * @var string
 */
protected $signature = 'email:send {user} {--queue}';
```

In this example, the `--queue` switch may be specified when calling the Artisan command. If the `--queue` switch is passed, the value of the option will be `true`. Otherwise, the value will be `false`:

```
php artisan email:send 1 --queue
```

You may also specify that the option should be assigned a value by the user by suffixing the option name with a `=` sign, indicating that a value should be provided:

```
/**
 * The name and signature of the console command.
 *
 * @var string
```

```
*/  
protected $signature = 'email:send {user} {--queue=}';
```

In this example, the user may pass a value for the option like so:

```
php artisan email:send 1 --queue=default
```

You may also assign default values to options:

```
email:send {user} {--queue=default}
```

To assign a shortcut when defining an option, you may specify it before the option name and use a | delimiter to separate the shortcut from the full option name:

```
email:send {user} {--Q|queue}
```

Input Descriptions

You may assign descriptions to input arguments and options by separating the parameter from the description using a colon:

```
/**  
 * The name and signature of the console command.  
 *  
 * @var string  
 */  
protected $signature = 'email:send  
                        {user : The ID of the user} {--queue=  
                        : Whether the job should be  
                        queued}';
```

Retrieving Input

While your command is executing, you will obviously need to access the values for the arguments and options accepted by your command. To do so, you may use the `argument` and `option` methods:

To retrieve the value of an argument, use the `argument` method:

```
/**  
 * Execute the console command.  
 *  
 * @return mixed  
 */  
public function handle()  
{  
    $userId = $this->argument('user');  
  
    //  
}
```

If you need to retrieve all of the arguments as an array, call `argument` with no parameters:

```
$arguments = $this->argument();
```

Options may be retrieved just as easily as arguments using the `option` method. Like the `argument` method, you may call `option` without any parameters in order to retrieve all of the options as an array:

```
// Retrieve a specific option...
$queueName = $this->option('queue');

// Retrieve all options...
$options = $this->option();
```

If the argument or option does not exist, `null` will be returned.

Prompting For Input

In addition to displaying output, you may also ask the user to provide input during the execution of your command. The `ask` method will prompt the user with the given question, accept their input, and then return the user's input back to your command:

```
/**
 * Execute the console command.
 *
 * @return mixed
 */
public function handle()
{
    $name = $this->ask('What is your name?');
}
```

The `secret` method is similar to `ask`, but the user's input will not be visible to them as they type in the console. This method is useful when asking for sensitive information such as a password:

```
$password = $this->secret('What is the password?');
```

Asking For Confirmation

If you need to ask the user for a simple confirmation, you may use the `confirm` method. By default, this method will return `false`. However, if the user enters `y` in response to the prompt, the method will return `true`.

```
if ($this->confirm('Do you wish to continue? [y|N]')) {
    //
}
```

Giving The User A Choice

The `anticipate` method can be used to provided autocompletion for possible choices. The user can still choose any answer, regardless of the choices.

```
$name = $this->anticipate('What is your name?', ['Taylor',  
'Dayle']);
```

If you need to give the user a predefined set of choices, you may use the `choice` method. The user chooses the index of the answer, but the value of the answer will be returned to you. You may set the default value to be returned if nothing is chosen:

```
$name = $this->choice('What is your name?', ['Taylor',  
'Dayle'], false);
```

Writing Output

To send output to the console, use the `line`, `info`, `comment`, `question` and `error` methods. Each of these methods will use the appropriate ANSI colors for their purpose.

To display an information message to the user, use the `info` method. Typically, this will display in the console as green text:

```
/**  
 * Execute the console command.  
 */  
 * @return mixed  
 */  
public function handle()  
{  
    $this->info('Display this on the screen');  
}
```

To display an error message, use the `error` method. Error message text is typically displayed in red:

```
$this->error('Something went wrong!');
```

If you want to display plain console output, use the `line` method. The `line` method does not receive any unique coloration:

```
$this->line('Display this on the screen');
```

Table Layouts

The `table` method makes it easy to correctly format multiple rows / columns of data. Just pass in the headers and rows to the method. The width and height will be dynamically calculated based on the given data:

```
$headers = ['Name', 'Email'];  
  
$users = App\User::all(['name', 'email'])->toArray();  
  
$this->table($headers, $users);
```

Progress Bars

For long running tasks, it could be helpful to show a progress indicator. Using the output object, we can start, advance and stop the Progress Bar. You have to define the number of steps when you start the progress, then advance the Progress Bar after each step:

```
$users = App\User::all();

$bar = $this->output->createProgressBar(count($users));

foreach ($users as $user) {
    $this->performTask($user);

    $bar->advance();
}

$bar->finish();
```

For more advanced options, check out the [Symfony Progress Bar component documentation](#).

REGISTERING COMMANDS

Once your command is finished, you need to register it with Artisan so it will be available for use. This is done within the `app/Console/Kernel.php` file.

Within this file, you will find a list of commands in the `commands` property. To register your command, simply add the class name to the list. When Artisan boots, all the commands listed in this property will be resolved by the [service container](#) and registered with Artisan:

```
protected $commands = [
    Commands\SendEmails::class
];
```

CALLING COMMANDS VIA CODE

Sometimes you may wish to execute an Artisan command outside of the CLI. For example, you may wish to fire an Artisan command from a route or controller. You may use the `call` method on the `Artisan` facade to accomplish this. The `call` method accepts the name of the command as the first argument, and an array of command parameters as the second argument. The exit code will be returned:

```
Route::get('/foo', function () {
    $exitCode = Artisan::call('email:send', [
        'user' => 1, '--queue' => 'default'
    ]);

    //
});
```

Using the `queue` method on the `Artisan` facade, you may even queue Artisan commands so they are processed in the background by your [queue workers](#):

```
Route::get('/foo', function () {
    Artisan::queue('email:send', [
        'user' => 1, '--queue' => 'default'
    ]);

    //
});
```

If you need to specify the value of an option that does not accept string values, such as the `--force` flag on the `migrate:refresh` command, you may pass a boolean `true` or `false`:

```
$exitCode = Artisan::call('migrate:refresh', [
    '--force' => true,
]);
```

Calling Commands From Other Commands

Sometimes you may wish to call other commands from an existing Artisan command. You may do so using the `call` method. This `call` method accepts the command name and an array of command parameters:

```
/**
 * Execute the console command.
 *
 * @return mixed
 */
public function handle()
{
    $this->call('email:send', [
        'user' => 1, '--queue' => 'default'
    ]);

    //
}
```

If you would like to call another console command and suppress all of its output, you may use the `callSilent` method. The `callSilent` method has the same signature as the `call` method:

```
$this->callSilent('email:send', [ 'user'
    => 1, '--queue' => 'default'
]);
```

😊 Database creation and artisan migration:

INTRODUCTION

Migrations are like version control for your database, allowing a team to easily modify and share the application's database schema. Migrations are typically paired with Laravel's schema builder to easily build your application's database schema.

The Laravel Schema facade provides database agnostic support for creating and manipulating tables. It shares the same expressive, fluent API across all of Laravel's supported database systems.

GENERATING MIGRATIONS / CREATING MIGRATION

To create a migration, use the `make:migration` Artisan command:

```
php artisan make:migration create_users_table
```

The new migration will be placed in your `database/migrations` directory. Each migration file name contains a timestamp which allows Laravel to determine the order of the migrations.

The `--table` and `--create` options may also be used to indicate the name of the table and whether the migration will be creating a new table. These options simply pre-fill the generated migration stub file with the specified table:

```
php artisan make:migration add_votes_to_users_table --table=users
```

```
php artisan make:migration create_users_table --create=users
```

If you would like to specify a custom output path for the generated migration, you may use the `--path` option when executing the `make:migration` command. The provided path should be relative to your application's base path.

MIGRATION STRUCTURE

A migration class contains two methods: `up` and `down`. The `up` method is used to add new tables, columns, or indexes to your database, while the `down` method should simply reverse the operations performed by the `up` method.

Within both of these methods you may use the Laravel schema builder to expressively create and modify tables. To learn about all of the methods available on the Schema builder, check out its documentation. For example, let's look at a sample migration that creates a `flights` table:

```
<?php
```

```
use Illuminate\Database\Schema\Blueprint; use  
Illuminate\Database\Migrations\Migration;
```



```
class CreateFlightsTable extends Migration
{
    /**
     * Run the migrations.
     *
     * @return
     * void */
    public function up()
    {
        Schema::create('flights', function (Blueprint $table) {
            $table->increments('id');
            $table->string('name');
            $table->string('airline');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return
     * void */
    public function down()
    {
        Schema::drop('flights');
    }
}
```

RUNNING MIGRATIONS

To run all outstanding migrations for your application, use the `migrate` Artisan command. If you are using the Homestead virtual machine, you should run this command from within your VM:

```
php artisan migrate
```

If you receive a "class not found" error when running migrations, try running the `composer dump-autoload` command and re-issuing the migrate command.

Forcing Migrations To Run In Production

Some migration operations are destructive, meaning they may cause you to lose data. In order to protect you from running these commands against your production database, you will be prompted for confirmation before these commands are executed. To force the commands to run without a prompt, use the `--force` flag:

```
php artisan migrate --force
```

ROLLING BACK MIGRATIONS

To rollback the latest migration "operation", you may use the `rollback` command. Note that this rolls back the last "batch" of migrations that ran, which may include multiple migration files:

```
php artisan migrate:rollback
```

The `migrate:reset` command will roll back all of your application's migrations:

```
php artisan migrate:reset
```

Rollback / Migrate In Single Command

The `migrate:refresh` command will first roll back all of your database migrations, and then run the `migrate` command. This command effectively re-creates your entire database:

```
php artisan migrate:refresh
```

```
php artisan migrate:refresh --seed
```

WRITING MIGRATIONS

Creating Tables

To create a new database table, use the `create` method on the `Schema` facade. The `create` method accepts two arguments. The first is the name of the table, while the second is a Closure which receives a `Blueprint` object used to define the new table:

```
Schema::create('users', function (Blueprint $table) {  
    $table->increments('id');  
});
```

Of course, when creating the table, you may use any of the schema builder's column methods to define the table's columns.

Checking For Table / Column Existence

You may easily check for the existence of a table or column using the `hasTable` and `hasColumn` methods:

```
if (Schema::hasTable('users')) {  
    //  
}  
  
if (Schema::hasColumn('users', 'email')) {  
    //  
}
```

Connection & Storage Engine

If you want to perform a schema operation on a database connection that is not your default connection, use the `connection` method:

```
Schema::connection('foo')->create('users', function ($table) {  
    $table->increments('id');  
});
```

To set the storage engine for a table, set the `engine` property on the schema builder:

```
Schema::create('users', function ($table) {  
    $table->engine = 'InnoDB';  
  
    $table->increments('id');  
});
```

RENAMING / DROPPING TABLES

To rename an existing database table, use the `rename` method:

```
Schema::rename($from, $to);
```

To drop an existing table, you may use the `drop` or `dropIfExists` methods:

```
Schema::drop('users');
```

```
Schema::dropIfExists('users');
```

CREATING COLUMNS

To update an existing table, we will use the `table` method on the `Schema` facade. Like the `create` method, the `table` method accepts two arguments: the name of the table and a Closure that receives a `Blueprint` instance we can use to add columns to the table:

```
Schema::table('users', function ($table) {  
    $table->string('email');  
});
```

Available Column Types

Of course, the schema builder contains a variety of column types that you may use when building your tables:

Command	Description
<code>\$table->bigIncrements('id');</code>	Incrementing ID (primary key) using a "UNSIGNED BIG INTEGER" equivalent.
<code>\$table->bigInteger('votes');</code>	BIGINT equivalent for the <u>database</u> .

Command	Description
<code>\$table->binary('data');</code>	BLOB equivalent for the database .
<code>>boolean('confirmed');</code>	BOOLEAN equivalent for the database .
<code>>char('name', 4);</code>	CHAR equivalent with a length.
<code>>date('created_at');</code>	DATE equivalent for the database .
<code>>dateTime('created_at');</code>	DATETIME equivalent for the database .
<code>>decimal('amount', 5, 2);</code>	DECIMAL equivalent with a precision and scale.
<code>\$table->double('column', 15, 8);</code>	DOUBLE equivalent with precision, 15 digits in total and 8 after the decimal point.
<code>\$table->enum('choices', ['foo', 'bar']);</code>	ENUM equivalent for the database .
<code>\$table->float('amount');</code>	FLOAT equivalent for the database .
<code>\$table->increments('id');</code>	Incrementing ID (primary key) using a "UNSIGNED INTEGER" equivalent.
<code>\$table->integer('votes');</code>	INTEGER equivalent for the database .
<code>>json('options');</code>	JSON equivalent for the database .
<code>>jsonb('options');</code>	JSONB equivalent for the database .
<code>>longText('description');</code>	LONGTEXT equivalent for the database .
<code>>mediumInteger('numbers');</code>	MEDIUMINT equivalent for the database .
<code>\$table->mediumText('description');</code>	MEDIUMTEXT equivalent for the database .
<code>\$table->morphs('taggable');</code>	Adds <code>INTEGER taggable_id</code> and <code>STRING taggable_type</code> .
<code>\$table->nullableTimestamps();</code>	Same as <code>timestamps()</code> , except allows NULLs.
<code>\$table->rememberToken();</code>	Adds <code>remember_token</code> as <code>VARCHAR(100) NULL</code> .
<code>\$table->smallInteger('votes');</code>	SMALLINT equivalent for the database .
<code>\$table->softDeletes();</code>	Adds <code>deleted_at</code> column for soft deletes.
<code>>string('email');</code>	VARCHAR equivalent column.
<code>>string('name', 100);</code>	VARCHAR equivalent with a length.

Command	Description
<code>\$table->text('description');</code>	TEXT equivalent for the database .
<code>\$table->time('sunrise');</code>	TIME equivalent for the database .
<code>\$table->tinyInteger('numbers');</code>	TINYINT equivalent for the database .
<code>\$table->timestamp('added_on');</code>	TIMESTAMP equivalent for the database .
<code>\$table->timestamps();</code>	Adds <code>created_at</code> and <code>updated_at</code> columns.
<code>\$table->uuid('id');</code>	UUID equivalent for the database .

Column Modifiers

In addition to the column types listed above, there are several other column "modifiers" which you may use while adding the column. For example, to make the column "nullable", you may use the `nullable` method:

```
Schema::table('users', function ($table) {
    $table->string('email')->nullable();
});
```

Below is a list of all the available column modifiers. This list does not include the [index modifiers](#):

Modifier	Description
<code>->first()</code>	Place the column "first" in the table (MySQL Only)
<code>->after('column')</code>	Place the column "after" another column (MySQL Only)
<code>->nullable()</code>	Allow NULL values to be inserted into the column
<code>->default(\$value)</code>	Specify a "default" value for the column
<code>->unsigned()</code>	Set integer columns to UNSIGNED

MODIFYING COLUMNS

Prerequisites

Before modifying a column, be sure to add the `doctrine/dbal` dependency to your `composer.json` file. The Doctrine DBAL library is used to determine the current state of the column and create the SQL queries needed to make the specified adjustments to the column.

Updating Column Attributes

The `change` method allows you to modify an existing column to a new type, or modify the column's attributes. For example, you may wish to increase the size of a string column. To see the `change` method in action, let's increase the size of the `name` column from 25 to 50:

```
Schema::table('users', function ($table) {  
    $table->string('name', 50)->change();  
});
```

We could also modify a column to be nullable:

```
Schema::table('users', function ($table) { $table->  
    >string('name', 50)->nullable()->change();  
});
```

Renaming Columns

To rename a column, you may use the `renameColumn` method on the Schema builder. Before renaming a column, be sure to add the `doctrine/dbal` dependency to your `composer.json` file:

```
Schema::table('users', function ($table) {  
    $table->renameColumn('from', 'to');  
});
```

Note: Renaming columns in a table with a `enum` column is not currently supported.

DROPPING COLUMNS

To drop a column, use the `dropColumn` method on the Schema builder:

```
Schema::table('users', function ($table) {  
    $table->dropColumn('votes');  
});
```

You may drop multiple columns from a table by passing an array of column names to the `dropColumn` method:

```
Schema::table('users', function ($table) { $table->  
    >dropColumn(['votes', 'avatar', 'location']);  
});
```

Note: Before dropping columns from a SQLite database, **you** will need to add the `doctrine/dbal` dependency to your `composer.json` file and run the `composer update` command in your terminal to install the library.

Note: Dropping or modifying multiple columns within a single migration while using a SQLite database is not supported.

CREATING INDEXES

The schema builder supports several types of indexes. First, let's look at an example that specifies a column's values should be unique. To create the index, we can simply chain the `unique` method onto the column definition:

```
$table->string('email')->unique();
```

Alternatively, you may create the index after defining the column. For example:

```
$table->unique('email');
```

You may even pass an array of columns to an index method to create a compound index:

```
$table->index(['account_id', 'created_at']);
```

Available Index Types

Command	Description
<code>\$table->primary('id');</code>	Add a primary key.
<code>\$table->primary(['first', 'last']);</code>	Add composite keys.
<code>\$table->unique('email');</code>	Add a unique index.
<code>\$table->index('state');</code>	Add a basic index.

DROPPING INDEXES

To drop an index, you must specify the index's name. By default, [Laravel](#) automatically assigns a reasonable name to the indexes. Simply concatenate the table name, the name of the indexed column, and the index type. Here are some examples:

Command	Description
<code>\$table->dropPrimary('users_id_primary');</code>	Drop a primary key from the "users" table.
<code>\$table->dropUnique('users_email_unique');</code>	Drop a unique index from the "users" table.
<code>\$table->dropIndex('geo_state_index');</code>	Drop a basic index from the "geo" table.

If you pass an array of columns into a method that drops indexes, the conventional index name will be generated based on the table name, columns and key type.

```
Schema::table('geo', function ($table) { $table->dropIndex(['state']); // Drops index 'geo_state_index'
```

```
});
```

FOREIGN KEY CONSTRAINTS

Laravel also provides support for creating foreign key constraints, which are used to force referential integrity at the database level. For example, let's define a `user_id` column on the `posts` table that references the `id` column on a `users` table:

```
Schema::table('posts', function ($table) {  
    $table->integer('user_id')->unsigned();  
  
    $table->foreign('user_id')->references('id')->on('users');  
});
```

You may also specify the desired action for the "on delete" and "on update" properties of the constraint:

```
$table->foreign('user_id') -  
    >references('id')->on('users')  
    ->onDelete('cascade');
```

To drop a foreign key, you may use the `dropForeign` method. Foreign key constraints use the same naming convention as indexes. So, we will concatenate the table name and the columns in the constraint then suffix the name with `"_foreign"`:

```
$table->dropForeign('posts_user_id_foreign');
```

Or you may pass an array value which will automatically use the conventional constraint name when dropping:

```
$table->dropForeign(['user_id']);
```


☺ Database seeding:

INTRODUCTION

Laravel includes a simple method of seeding your database with test data using seed classes. All seed classes are stored in `database/seeds`. Seed classes may have any name you wish, but probably should follow some sensible convention, such as `UsersTableSeeder`, etc. By default, a `DatabaseSeeder` class is defined for you. From this class, you may use the `call` method to run other seed classes, allowing you to control the seeding order.

WRITING SEEDERS

To generate a seeder, you may issue the `make:seeder` Artisan command. All seeders generated by the framework will be placed in the `database/seeds` directory:

```
php artisan make:seeder UsersTableSeeder
```

A seeder class only contains one method by default: `run`. This method is called when the `db:seed` Artisan command is executed. Within the `run` method, you may insert data into your database however you wish. You may use the query builder to manually insert data or you may use Eloquent model factories.

As an example, let's modify the `DatabaseSeeder` class which is included with a default installation of Laravel. Let's add a database insert statement to the `run` method:

```
<?php

use Illuminate\Database\Seeder;
use Illuminate\Database\Eloquent\Model;

class DatabaseSeeder extends Seeder
{
    /**
     * Run the database seeds.
     *
     * @return void
     */
    public function run()
    {
        DB::table('users')->insert([ 'name'
            => str_random(10),
            'email' => str_random(10).'@gmail.com',
            'password' => bcrypt('secret'),
        ]);
    }
}
```

USING MODEL FACTORIES

Of course, manually specifying the attributes for each model seed is cumbersome. Instead, you can use model factories to conveniently generate large amounts of database records.

First, review the [model factory documentation](#) to learn how to define your factories. Once you have defined your factories, you may use the `factory` helper function to insert records into your [database](#).

For example, let's create 50 users and attach a relationship to each user:

```
/**
 * Run the database seeds.
 *
 * @return void
 */
public function run()
{
    factory(App\User::class, 50)->create()->each(function($u)
    {
        $u->posts()->save(factory(App\Post::class)->make());
    });
}
```

CALLING ADDITIONAL SEEDERS

Within the `DatabaseSeeder` class, you may use the `call` method to execute additional seed classes. Using the `call` method allows you to break up your [database seeding](#) into multiple files so that no single seeder class becomes overwhelmingly large. Simply pass the name of the seeder class you wish to run:

```
/**
 * Run the database seeds.
 *
 * @return void
 */
public function run()
{
    Model::unguard();

    $this->call(UsersTableSeeder::class);
    $this->call(PostsTableSeeder::class);
    $this->call(CommentsTableSeeder::class);

    Model::reguard();
}
```

RUNNING SEEDERS

Once you have written your seeder classes, you may use the `db:seed` Artisan command to seed your [database](#). By default, the `db:seed` command runs the `DatabaseSeeder` class,

which may be used to call other seed classes. However, you may use the

option to

specify a specific seeder class to run individually:

`--class`

`php artisan db:seed`

`php artisan db:seed --class=UserTableSeeder`

You may also seed your database using the `migrate:refresh` command, which will also rollback and re-run all of your migrations. This command is useful for completely re-building your database:

```
php artisan migrate:refresh --seed
```

Chapter – 3.2 [Routing in Laravel]

☺ Basic Routing:

You will define most of the routes for your application in the `app/Http/routes.php` file, which is loaded by the `App\Providers\RouteServiceProvider` class. The most basic Laravel routes simply accept a URI and a Closure:

```
Route::get('/', function ()
    { return 'Hello World';
});

Route::post('foo/bar', function () {
    return 'Hello World';
});

Route::put('foo/bar', function () {
    //
});

Route::delete('foo/bar', function () {
    //
});
```

Registering A Route For Multiple Verbs

Sometimes you may need to register a route that responds to multiple HTTP verbs. You may do so using the `match` method on the Route facade:

```
Route::match(['get', 'post'], '/', function () {
    return 'Hello World';
});
```

Or, you may even register a route that responds to all HTTP verbs using the `any` method:

```
Route::any('foo', function () {
    return 'Hello World';
});
```

Generating URLs To Routes

You may generate URLs to your application's routes using the `url` helper:

```
$url = url('foo');
```

☺ Route Parameters:

REQUIRED PARAMETERS

Of course, sometimes you will need to capture segments of the URI within your route. For example, you may need to capture a user's ID from the URL. You may do so by defining route parameters:

```
Route::get('user/{id}', function ($id) {  
    return 'User '.$id;  
});
```

You may define as many route parameters as required by your route:

```
Route::get('posts/{post}/comments/{comment}', function  
($postId, $commentId) {  
    //  
});
```

Route parameters are always encased within "curly" braces. The parameters will be passed into your route's `Closure` when the route is executed.

Note: Route parameters cannot contain the `-` character. Use an underscore (`_`) instead.

OPTIONAL PARAMETERS

Occasionally you may need to specify a route parameter, but make the presence of that route parameter optional. You may do so by placing a `?` mark after the parameter name:

```
Route::get('user/{name?}', function ($name = null) {  
    return $name;  
});  
  
Route::get('user/{name?}', function ($name = 'John')  
{ return $name;  
});
```

REGULAR EXPRESSION CONSTRAINTS

You may constrain the format of your route parameters using the `where` method on a route instance. The `where` method accepts the name of the parameter and a regular expression defining how the parameter should be constrained:

```
Route::get('user/{name}', function ($name) {  
    //  
})  
->where('name', '[A-Za-z]+');
```

```
Route::get('user/{id}', function ($id) {  
    //  
})
```

```
->where('id', '[0-9]+');

Route::get('user/{id}/{name}', function ($id, $name) {
    //
})
->where(['id' => '[0-9]+', 'name' => '[a-z]+']);
```

Global Constraints

If you would like a route parameter to always be constrained by a given regular expression, you may use the `pattern` method. You should define these patterns in the `boot` method of your `RouteServiceProvider`:

```
/**
 * Define your route model bindings, pattern filters, etc.
 *
 * @param \Illuminate\Routing\Router $router
 * @return void
 */
public function boot(Router $router)
{
    $router->pattern('id', '[0-9]+');

    parent::boot($router);
}
```

Once the pattern has been defined, it is automatically applied to all routes using that parameter name:

```
Route::get('user/{id}', function ($id) {
    // Only called if {id} is numeric.
});
```

☺ Named Route:

Named routes allow you to conveniently generate URLs or redirects for a specific route. You may specify a name for a route using the `as` array key when defining the route:

```
Route::get('user/profile', ['as' => 'profile', function () {
    //
}]);
```

You may also specify route names for controller actions:

```
Route::get('user/profile', [
    'as' => 'profile', 'uses' => 'UserController@showProfile'
]);
```

Instead of specifying the route name in the route array definition, you may chain the `name` method onto the end of the route definition:

```
Route::get('user/profile',  
'UserController@showProfile')->name('profile');
```

Route Groups & Named Routes

If you are using route groups, you may specify an `as` keyword in the route group attribute array, allowing you to set a common route name prefix for all routes within the group:

```
Route::group(['as' => 'admin::'], function () {  
    Route::get('dashboard', ['as' => 'dashboard', function () {  
        // Route named "admin::dashboard"  
    }]);  
});
```

Generating URLs To Named Routes

Once you have assigned a name to a given route, you may use the route's name when generating URLs or redirects via the `route` function:

```
$url = route('profile');  
  
$redirect = redirect()->route('profile');
```

If the route defines parameters, you may pass the parameters as the second argument to the `route` method. The given parameters will automatically be inserted into the URL:

```
Route::get('user/{id}/profile', ['as' => 'profile', function ($id) {  
    //  
}]);  
  
$url = route('profile', ['id' => 1]);
```

🔒 Route Groups:

Route groups allow you to share route attributes, such as middleware or namespaces, across a large number of routes without needing to define those attributes on each individual route. Shared attributes are specified in an array format as the first parameter to the `Route::group` method.

To learn more about route groups, we'll walk through several common use-cases for the feature.

MIDDLEWARE

To assign middleware to all routes within a group, you may use the `middleware` key in the group attribute array. Middleware will be executed in the order you define this array:

```
Route::group(['middleware' => 'auth'], function () {  
    Route::get('/', function () {
```

```
        // Uses Auth Middleware
    });

    Route::get('user/profile', function ()
    { // Uses Auth Middleware
    });
});
```

NAMESPACES

Another common use-case for route groups is assigning the same PHP namespace to a group of controllers. You may use the `namespace` parameter in your group attribute array to specify the namespace for all controllers within the group:

```
Route::group(['namespace' => 'Admin'], function()
{
    // Controllers Within The "App\Http\Controllers\Admin" Namespace

    Route::group(['namespace' => 'User'], function()
    {
        // Controllers Within The
        "App\Http\Controllers\Admin\User" Namespace
    });
});
```

Remember, by default, the `RouteServiceProvider` includes your `routes.php` file within a namespace group, allowing you to register controller routes without specifying the full `App\Http\Controllers` namespace prefix. So, we only need to specify the portion of the namespace that comes after the base `App\Http\Controllers` namespace root.

☺ Sub-Domain Routing:

Route groups may also be used to route wildcard sub-domains. Sub-domains may be assigned route parameters just like route URIs, allowing you to capture a portion of the sub-domain for usage in your route or controller. The sub-domain may be specified using the `domain` key on the group attribute array:

```
Route::group(['domain' => '{account}.myapp.com'], function () {
    Route::get('user/{id}', function ($account, $id) {
        //
    });
});
```


☺ Route Prefix:

The `prefix` group array attribute may be used to prefix each route in the group with a given URI. For example, you may want to prefix all route URIs within the group with `admin`:

```
Route::group(['prefix' => 'admin'], function () {
    { Route::get('users', function () {
        // Matches The "/admin/users" URL
    });
});
```

You may also use the `prefix` parameter to specify common parameters for your grouped routes:

```
Route::group(['prefix' => 'accounts/{account_id}'], function () {
    Route::get('detail', function ($account_id) {
        // Matches The accounts/{account_id}/detail URL
    });
});
```

☺ Route Model Binding:

Laravel route model binding provides a convenient way to inject class instances into your routes. For example, instead of injecting a user's ID, you can inject the entire `User` class instance that matches the given ID.

First, use the router's `model` method to specify the class for a given parameter. You should define your model bindings in the `RouteServiceProvider::boot` method:

Binding A Parameter To A Model

```
public function boot(Router $router)
{
    parent::boot($router);

    $router->model('user', 'App\User');
}
```

Next, define a route that contains a `{user}` parameter:

```
$router->get('profile/{user}', function(App\User $user) {
    //
});
```

Since we have bound the `{user}` parameter to the `App\User` model, a `User` instance will be injected into the route. So, for example, a request to `profile/1` will inject the `User` instance which has an ID of 1.

☹ Throwing 404 Errors:

Note: If a matching model instance is not found in the database, a 404 exception will be thrown automatically.

If you wish to specify your own "not found" behavior, pass a Closure as the third argument to the `model` method:

```
$router->model('user', 'App\User', function() {  
    throw new NotFoundHttpException;  
});
```

If you wish to use your own resolution logic, you should use the `Route::bind` method. The Closure you pass to the `bind` method will receive the value of the URI segment, and should return an instance of the class you want to be injected into the route:

```
$router->bind('user', function($value) {  
    return App\User::where('name', $value)->first();  
});
```

There are two ways to manually trigger a 404 error from a route. First, you may use the `abort` helper. The `abort` helper simply throws a `Symfony\Component\HttpFoundation\Exception\HttpException` with the specified status code:

```
abort(404);
```

Secondly, you may manually throw an instance of `Symfony\Component\HttpKernel\Exception\NotFoundHttpException`.

More information on handling 404 exceptions and using custom responses for these errors may be found in the [errors](#) section of the documentation.

☺ Routing to Controller:

INTRODUCTION

Instead of defining all of your request handling logic in a single `routes.php` file, you may wish to organize this behavior using Controller classes. Controllers can group related HTTP request handling logic into a class. Controllers are typically stored in the `app/Http/Controllers` directory.

BASIC CONTROLLERS

Here is an example of a basic controller class. All Laravel controllers should extend the base controller class included with the default Laravel installation:

```
<?php

namespace App\Http\Controllers;

use App\User;
use App\Http\Controllers\Controller;

class UserController extends Controller
{
    /**
     * Show the profile for the given user.
     *
     * @param int $id *
     * @return Response
     */
    public function showProfile($id)
    {
        return view('user.profile', ['user' =>
User::findOrFail($id)]);
    }
}
```

We can route to the controller action like so:

```
Route::get('user/{id}', 'UserController@showProfile');
```

Now, when a request matches the specified route URI, the `showProfile` method on the `UserController` class will be executed. Of course, the route parameters will also be passed to the method.

Controllers & Namespaces

It is very important to note that we did not need to specify the full controller namespace when defining the controller route. We only defined the portion of the class name that comes after the `App\Http\Controllers` namespace "root". By default, the `RouteServiceProvider` will load the `routes.php` file within a route group containing the root controller namespace.

If you choose to nest or organize your controllers using PHP namespaces deeper into the `App\Http\Controllers` directory, simply use the specific class name relative to the `App\Http\Controllers` root namespace. So, if your full controller class is `App\Http\Controllers\Photos\AdminController`, you would register a route like so:

```
Route::get('foo', 'Photos\AdminController@method');
```

Naming Controller Routes

Like Closure routes, you may specify names on controller routes:

```
Route::get('foo', ['uses' => 'FooController@method', 'as' => 'name']);
```

URLs To Controller Actions

You may also use the `route` helper to generate a URL to a named controller route:

```
$url = route('name');
```

You may also use the `action` helper method to generate a URL using the controller's class and method names. Again, we only need to specify the part of the controller class name that comes after the base `App\Http\Controllers` namespace:

```
$url = action('FooController@method');
```

You may access the name of the controller action being run using the `currentRouteAction` method on the `Route` facade:

```
$action = Route::currentRouteAction();
```

CONTROLLER MIDDLEWARE

Middleware may be assigned to the controller's routes like so:

```
Route::get('profile', [
    'middleware' => 'auth',
    'uses' => 'UserController@showProfile'
]);
```

However, it is more convenient to specify middleware within your controller's constructor. Using the `middleware` method from your controller's constructor, you may easily assign middleware to the controller. You may even restrict the middleware to only certain methods on the controller class:

```
class UserController extends Controller
{
    /**
     * Instantiate a new UserController instance.
     */
}
```

```
* @return
void */
public function __construct()
{
    $this->middleware('auth');

    $this->middleware('log', ['only' =>
['fooAction', 'barAction']]);

    $this->middleware('subscribed', ['except' => ['fooAction',
'barAction']]);
}
}
```

RESTful Resource Controllers

Resource controllers make it painless to build RESTful controllers around resources. For example, you may wish to create a controller that handles HTTP requests regarding "photos" stored by your application. Using the `make:controller` Artisan command, we can quickly create such a controller:

```
php artisan make:controller PhotoController
```

The Artisan command will generate a controller file at `app/Http/Controllers/PhotoController.php`. The controller will contain a method for each of the available resource operations.

Next, you may register a resourceful route to the controller:

```
Route::resource('photo', 'PhotoController');
```

This single route declaration creates multiple routes to handle a variety of RESTful actions on the photo resource. Likewise, the generated controller will already have methods stubbed for each of these actions, including notes informing you which URIs and verbs they handle.

Actions Handled By Resource Controller

Verb	Path	Action	Route Name
GET	/photo	index	photo.index
GET	/photo/create	create	photo.create
POST	/photo	store	photo.store
GET	/photo/{photo}	show	photo.show
GET	/photo/{photo}/edit	edit	photo.edit
PUT/PATCH	/photo/{photo}	update	photo.update
DELETE	/photo/{photo}	destroy	photo.destroy

Partial Resource Routes

When declaring a resource route, you may specify a subset of actions to handle on the route:

```
Route::resource('photo', 'PhotoController',
    ['only' => ['index', 'show']]);

Route::resource('photo', 'PhotoController',
    ['except' => ['create', 'store', 'update',
'destroy']]);
```

Naming Resource Routes

By default, all resource controller actions have a route name; however, you can override these names by passing a `names` array with your options:

```
Route::resource('photo', 'PhotoController',
    ['names' => ['create' => 'photo.build']]);
```

Nested Resources

Sometimes you may need to define routes to a "nested" resource. For example, a photo resource may have multiple "comments" that may be attached to the photo. To "nest" resource controllers, use "dot" notation in your route declaration:

```
Route::resource('photos.comments', 'PhotoCommentController');
```

This route will register a "nested" resource that may be accessed with URLs like the following: `photos/{photos}/comments/{comments}`.

```
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;

class PhotoCommentController extends Controller
{
    /**
     * Show the specified photo comment.
     *
     * @param int $photoId
     * @param int $commentId
     * @return Response
     */
    public function show($photoId, $commentId)
    {
        //
    }
}
```

Supplementing Resource Controllers

If it becomes necessary to add additional routes to a resource controller beyond the default resource routes, you should define those routes before your call to `Route::resource`; otherwise, the routes defined by the `resource` method may unintentionally take precedence over your supplemental routes:

```
Route::get('photos/popular', 'PhotoController@method');

Route::resource('photos', 'PhotoController');
```

IMPLICIT CONTROLLERS

Laravel allows you to easily define a single route to handle every action in a controller class. First, define the route using the `Route::controller` method. The `controller` method accepts two arguments. The first is the base URI the controller handles, while the second is the class name of the controller:

```
Route::controller('users', 'UserController');
```

Next, just add methods to your controller. The method names should begin with the HTTP verb they respond to followed by the title case version of the URI:

```
<?php

namespace App\Http\Controllers;

class UserController extends Controller
{
    /**
     * Responds to requests to GET /users
     */
    public function getIndex()
    {
        //
    }

    /**
     * Responds to requests to GET /users/show/1
     */
    public function getShow($id)
    {
        //
    }

    /**
     * Responds to requests to GET /users/admin-
     * profile */
    public function getAdminProfile()
    {
        //
    }
}
```

```
/**
 * Responds to requests to POST
 * /users/profile */
public function postProfile()
{
    //
}
```

As you can see in the example above, `index` methods will respond to the root URI handled by the controller, which, in this case, is `users`.

Assigning Route Names

If you would like to name some of the routes on the controller, you may pass an array of names as the third argument to the `controller` method:

```
Route::controller('users', 'UserController', [
    'getShow' => 'user.show',
]);
```

DEPENDENCY INJECTION & CONTROLLERS

Constructor Injection

The Laravel service container is used to resolve all Laravel controllers. As a result, you are able to type-hint any dependencies your controller may need in its constructor. The dependencies will automatically be resolved and injected into the controller instance:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Routing\Controller;
use App\Repositories\UserRepository;

class UserController extends Controller
{
    /**
     * The user repository instance.
     */
    protected $users;

    /**
     * Create a new controller instance.
     *
     * @param UserRepository $users
     * @return void
     */
    public function __construct(UserRepository $users)
    {
        $this->users = $users;
    }
}
```



```
}
```

Of course, you may also type-hint any Laravel contract. If the container can resolve it, you can type-hint it.

Method Injection

In addition to constructor injection, you may also type-hint dependencies on your controller's action methods. For example, let's type-hint the `Illuminate\Http\Request` instance on one of our methods:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Routing\Controller;

class UserController extends Controller
{
    /**
     * Store a new user.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        $name = $request->input('name');

        //
    }
}
```

If your controller method is also expecting input from a route parameter, simply list your route arguments after your other dependencies. For example, if your route is defined like so:

```
Route::put('user/{id}', 'UserController@update');
```

You may still type-hint the `Illuminate\Http\Request` and access your route parameter `id` by defining your controller method like the following:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Routing\Controller;

class UserController extends Controller
{
    /**
     * Update the specified user.
```

```
*
* @param Request $request
* @param int $id
* @return Response
*/
public function update(Request $request, $id)
{
    //
}
}
```

ROUTE CACHING

Note: Route caching does not work with Closure based routes. To use route caching, you must convert any Closure routes to use controller classes.

If your application is exclusively using controller based routes, you may take advantage of Laravel's route cache. Using the route cache will drastically decrease the amount of time it takes to register all of your application's routes. In some cases, your route registration may even be up to 100x faster! To generate a route cache, just execute the `route:cache` Artisan command:

```
php artisan route:cache
```

That's all there is to it! Your cached routes file will now be used instead of your `app/Http/routes.php` file. Remember, if you add any new routes you will need to generate a fresh route cache. Because of this, you may wish to only run the `route:cache` command during your project's deployment.

To remove the cached routes file without generating a new cache, use the `route:clear` command:

```
php artisan route:clear
```

☺ routing filter/route filter

Route filters provide a convenient way of limiting access to a given route, which is useful for creating areas of your site which require authentication. There are several filters included in the Laravel framework, including an `auth` filter, an `auth.basic` filter, a `guest` filter, and a `csrf` filter. These are located in the `app/filters.php` file.

Defining A Route Filter

```
Route::filter('old', function()
{
    if (Input::get('age') < 200)
    {
        return Redirect::to('home');
    }
});
```

❖ Routes, controllers etc related programs –

1. Write a program in laravel to display welcome page.
app/Http/routes.php

```
<?php
Route::get('/', function () {
    return view('welcome');
});
```

resources/view/welcome.blade.php

```
<!DOCTYPE html>
<html>

    <head>

        <title>Laravel</title>

        <link href = "https://fonts.googleapis.com/css?family=Lato:100" rel = "stylesheet"
            type = "text/css">

        <style>
            html, body {
                height: 100%;
            }
            body {
                margin: 0;
                padding: 0;
                width: 100%;
                display: table;
                font-weight: 100;
                font-family: 'Lato';
            }
            .container {
                text-align: center;
                display: table-cell;
                vertical-align: middle;
            }
            .content {
                text-align: center;
                display: inline-block;
            }
            .title {
                font-size: 96px;
```

```
    }  
    </style>  
</head>  
  
<body>  
    <div class = "container">  
  
        <div class = "content">  
            <div class = "title">Laravel </div>  
        </div>  
  
    </div>  
</body>  
  
</html>
```

Output:

Laravel

2. Write a program of laravel in which pass required parameter and option parameter

app/Http/routes.php

```
<?php  
  
// First Route method - Root URL will match this method  
Route::get('/', function () {  
    return view('welcome');  
});  
  
// Second Route method - Root URL with ID will match this method  
Route::get('ID/{id}',function($id){  
    echo 'ID: '.$id;  
});
```

```
// Third Route method - Root URL with or without name will match this method
Route::get('/user/{name?}',function($name = 'Virat Gandhi'){
    echo "Name: ".$name;
});
```

Step 1 – Here, we have defined 3 routes with get methods for different purposes. If we execute the below URL then it will execute the first method.

http://localhost:8000

After successful execution of the URL, you will receive the following output –

Laravel

Step 2– If we execute the below URL, it will execute the 2nd method and the argument/parameter ID will be passed to the variable \$id.

http://localhost:8000/ID/5

After successful execution of the URL, you will receive the following output

ID: 5

Step 3 – If we execute the below URL, it will execute the 3rd method and the optional argument/parameter name will be passed to the variable \$name. The last argument 'Virat' is optional. If you remove it, the default name will be used that we have passed in the function as 'Virat Gandhi'

http://localhost:8000/user/Virat

After successful execution of the URL, you will receive the following output

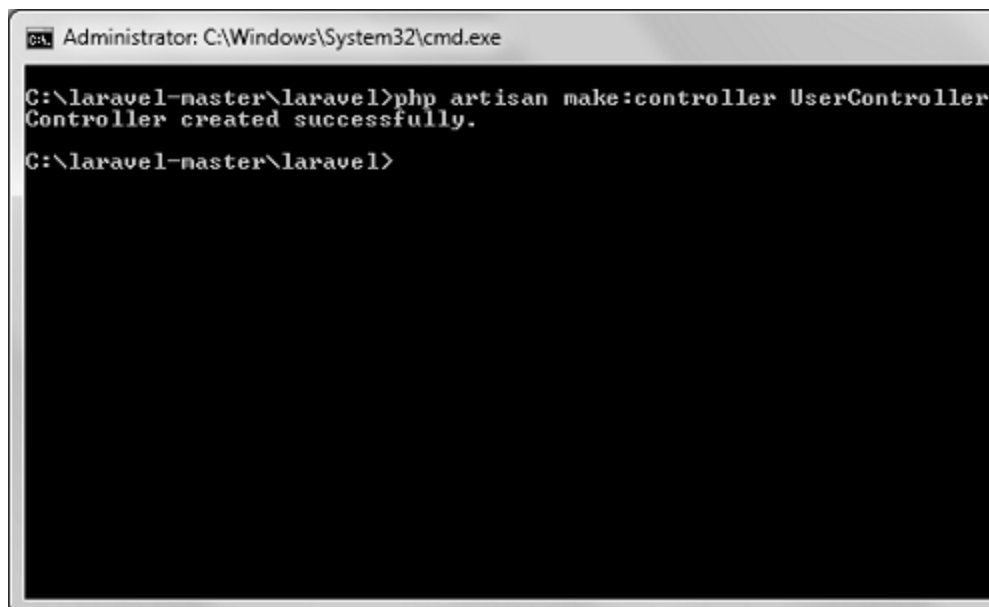
Name: Virat

3. Write command to create controller in laravel

Step 1 – Execute the following command to create **UserController**.

```
php artisan make:controller UserController
```

Step 2 – After successful execution, you will receive the following output.



```
Administrator: C:\Windows\System32\cmd.exe
C:\laravel-master\laravel>php artisan make:controller UserController
Controller created successfully.
C:\laravel-master\laravel>
```

Step 3 – You can see the created controller at **app/Http/Controller/UserController.php** with some basic coding already written for you and you can add your own coding based on your need.

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
use App\Http\Controllers\Controller;

class UserController extends Controller {
    //
}
```

4. Write a code to create controller and middleware in laravel and display url and method.

Step 1 – Add the following lines to the **app/Http/routes.php** file and save it.

routes.php

```
<?php
Route::get('/usercontroller/path',[
    'middleware' => 'First',
    'uses' => 'UserController@showPath'
]);
```

Step 2 – Create a middleware called **FirstMiddleware** by executing the following line.

```
php artisan make:middleware FirstMiddleware
```

Step 3 – Add the following code in the handle method of the newly created FirstMiddleware at **app/Http/Middleware**.

FirstMiddleware.php

```
<?php
namespace App\Http\Middleware;
use Closure;

class FirstMiddleware {
    public function handle($request, Closure $next) {
        echo '<br>First Middleware';
        return $next($request);
    }
}
```

Step 4 – Create a middleware called **SecondMiddleware** by executing the following line.

```
php artisan make:middleware SecondMiddleware
```

Step 5 – Add the following code in the handle method of the newly created SecondMiddleware at **app/Http/Middleware**.

SecondMiddleware.php

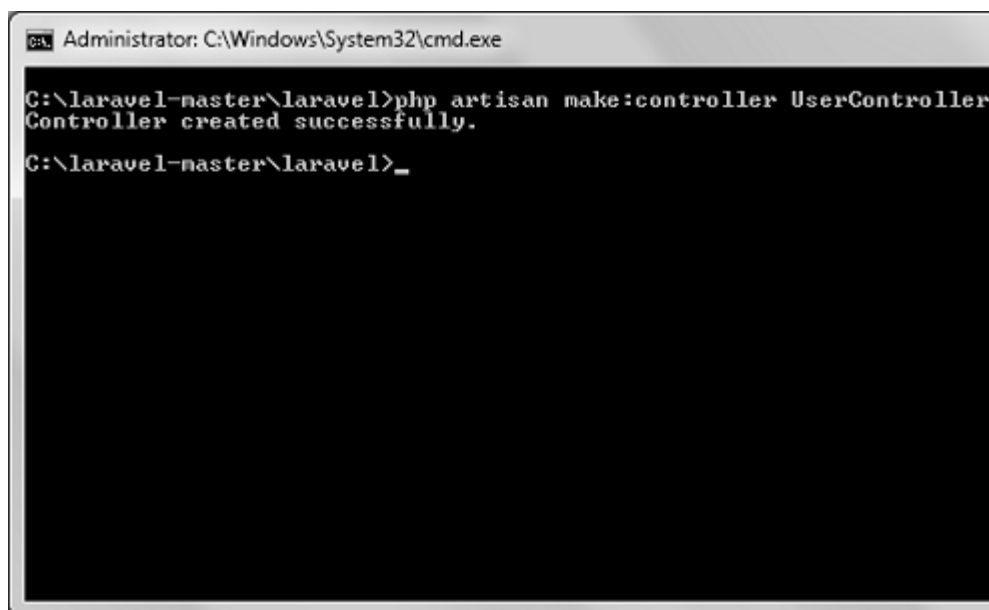
```
<?php
namespace App\Http\Middleware;
use Closure;
```

```
class SecondMiddleware {
    public function handle($request, Closure $next){
        echo '<br>Second Middleware';
        return $next($request);
    }
}
```

Step 6 – Create a controller called **UserController** by executing the following line.

```
php artisan make:controller UserController
```

Step 7 – After successful execution of the URL, you will receive the following output –



```
Administrator: C:\Windows\System32\cmd.exe
C:\laravel-master\laravel>php artisan make:controller UserController
Controller created successfully.
C:\laravel-master\laravel>_
```

Step 8 – Copy the following code to **app/Http/UserController.php** file.

app/Http/UserController.php

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
use App\Http\Controllers\Controller;

class UserController extends Controller {
    public function __construct(){
        $this->middleware('Second');
    }

    public function showPath(Request $request){
        $uri = $request->path();
        echo '<br>URI: '.$uri;
```



```

        $url = $request->url();
        echo '<br>';

        echo 'URL: '.$url;
        $method = $request->method();
        echo '<br>';

        echo 'Method: '.$method;
    }
}

```

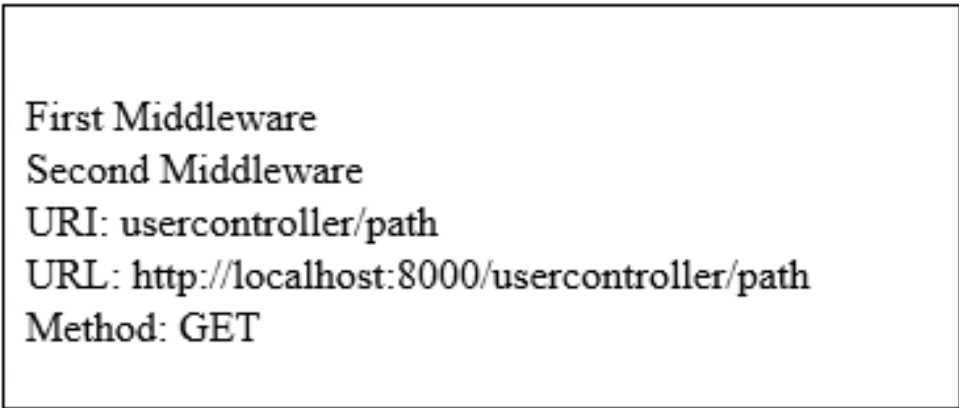
Step 9 – Now launch the php’s internal web server by executing the following command, if you haven’t executed it yet.

```
php artisan serve
```

Step 10 – Visit the following URL.

http://localhost:8000/usercontroller/path

Step 11 – The output will appear as shown in the following image.



```

First Middleware
Second Middleware
URI: usercontroller/path
URL: http://localhost:8000/usercontroller/path
Method: GET

```

5. Create controller in laravel and access class.

Step 1 – Create a controller called **MyController** by executing the following command.

```
php artisan make:controller MyController
```

Step 2 – Add the following code to **app/Http/routes.php** file.

app/Http/routes.php

```

class MyClass{
    public $foo = 'bar';
}

```

```
Route::get('/myclass','ImplicitController@index');
```

Step 3 – Add the following code to **app/Http/Controllers/ImplicitController.php** file.

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
use App\Http\Controllers\Controller;

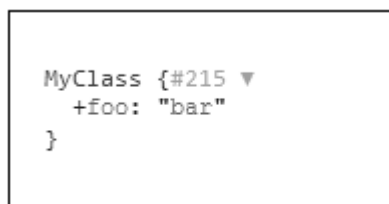
class ImplicitController extends Controller {
    private $myclass;

    public function __construct(\MyClass $myclass){
        $this->myclass = $myclass;
    }
    public function index(){
        dd($this->myclass);
    }
}
```

Step 4 – Visit the following URL to test the constructor injection.

<http://localhost:8000/myclass>

Step 5 – The output will appear as shown in the following image.



6. Write a laravel code to display validation in login form

Step 1 – Create a controller called **ValidationController** by executing the following command.

```
php artisan make:controller ValidationController
```

Step 2 – After successful execution, you will receive the following output –

```
Administrator: C:\Windows\System32\cmd.exe
C:\laravel-master\laravel>php artisan make:controller ValidationController
in
Controller created successfully.
C:\laravel-master\laravel>
```

Step 3 – Copy the following code in **app/Http/Controllers/ValidationController.php** file.

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
use App\Http\Controllers\Controller;

class ValidationController extends Controller {
    public function showform(){
        return view('login');
    }
    public function validateform(Request $request){
        print_r($request->all());
        $this->validate($request,[
            'username'=>'required|max:8',
            'password'=>'required'
        ]);
    }
}
```

Step 4 – Create a view file called **resources/views/login.blade.php** and copy the following code in that file.

```
<html>

<head>
    <title>Login Form</title>
</head>
```

```
<body>

@if (count($errors) > 0)
    <div class = "alert alert-danger">
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

<?php
    echo Form::open(array('url'=>'/validation'));
?>

<table border = '1'>
    <tr>
        <td align = 'center' colspan = '2'>Login</td>
    </tr>

    <tr>
        <td>Username</td>
        <td><?php echo Form::text('username'); ?></td>
    </tr>

    <tr>
        <td>Password</td>
        <td><?php echo Form::password('password'); ?></td>
    </tr>

    <tr>
        <td align = 'center' colspan = '2'>
            <?php echo Form::submit('Login'); ?> </td>
        </tr>
</table>

<?php
    echo Form::close();
?>

</body>
</html>
```

Step 5 – Add the following lines in **app/Http/routes.php**.

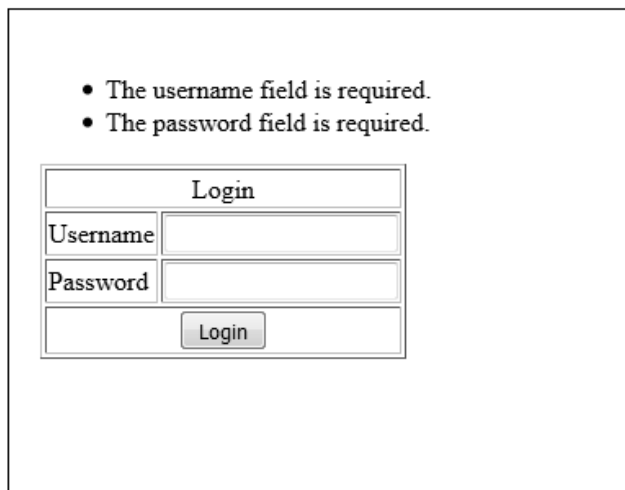
app/Http/routes.php

```
Route::get('/validation','ValidationController@showform');  
Route::post('/validation','ValidationController@validateform');
```

Step 6 – Visit the following URL to test the validation.

http://localhost:8000/validation

Step 7 – Click the “**Login**” button without entering anything in the text field. The output will be as shown in the following image.



• The username field is required.
• The password field is required.

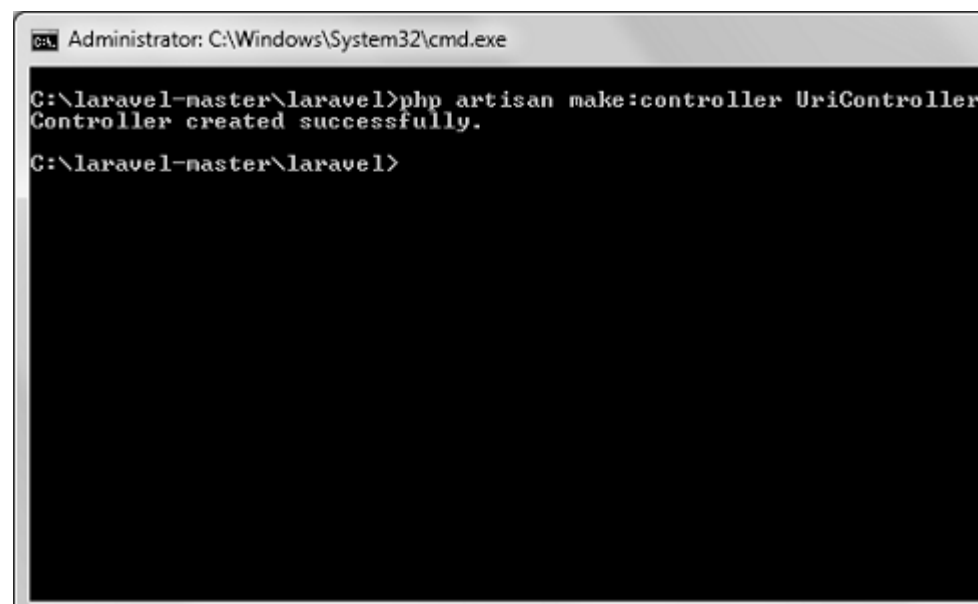
Login	
Username	
Password	
Login	

7. Write a code to display Request URL

Step 1 – Execute the below command to create a new controller called **UriController**.

```
php artisan make:controller UriController
```

Step 2 – After successful execution of the URL, you will receive the following output –



```
Administrator: C:\Windows\System32\cmd.exe  
  
C:\laravel-master\laravel>php artisan make:controller UriController  
Controller created successfully.  
C:\laravel-master\laravel>
```

Step 3 – After creating a controller, add the following code in that file.

app/Http/Controllers/UriController.php

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
use App\Http\Controllers\Controller;

class UriController extends Controller {

    public function index(Request $request){

        // Usage of path method
        $path = $request->path();
        echo 'Path Method: '.$path;
        echo '<br>';

        // Usage of is method
        $pattern = $request->is('foo/*');
        echo 'is Method: '.$pattern;
        echo '<br>';

        // Usage of url method
        $url = $request->url();
        echo 'URL method: '.$url;

    }
}
```

Step 4 – Add the following line in the **app/Http/route.php** file.

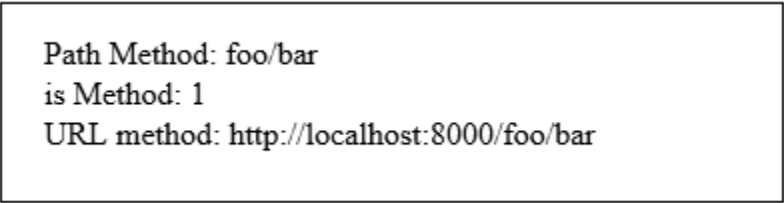
app/Http/route.php

```
Route::get('/foo/bar', 'UriController@index');
```

Step 5 – Visit the following URL.

http://localhost:8000/foo/bar

Step 6 – The output will appear as shown in the following image.



```
Path Method: foo/bar
is Method: 1
URL method: http://localhost:8000/foo/bar
```

8. Write a laravel code to display register form example.

Step 1 – Create a Registration form, where user can register himself and store the form at **resources/views/register.php**

```
<html>

<head>
    <title>Form Example</title>
</head>

<body>
    <form action = "/user/register" method = "post">
        <input type = "hidden" name = "_token" value = "<?php echo csrf_token()
?>">

        <table>
            <tr>
                <td>Name</td>
                <td><input type = "text" name = "name" /></td>
            </tr>

            <tr>
                <td>Username</td>
                <td><input type = "text" name = "username" /></td>
            </tr>

            <tr>
                <td>Password</td>
                <td><input type = "text" name = "password" /></td>
            </tr>

            <tr>
                <td colspan = "2" align = "center">
                    <input type = "submit" value = "Register" />
                </td>
            </tr>
        </table>

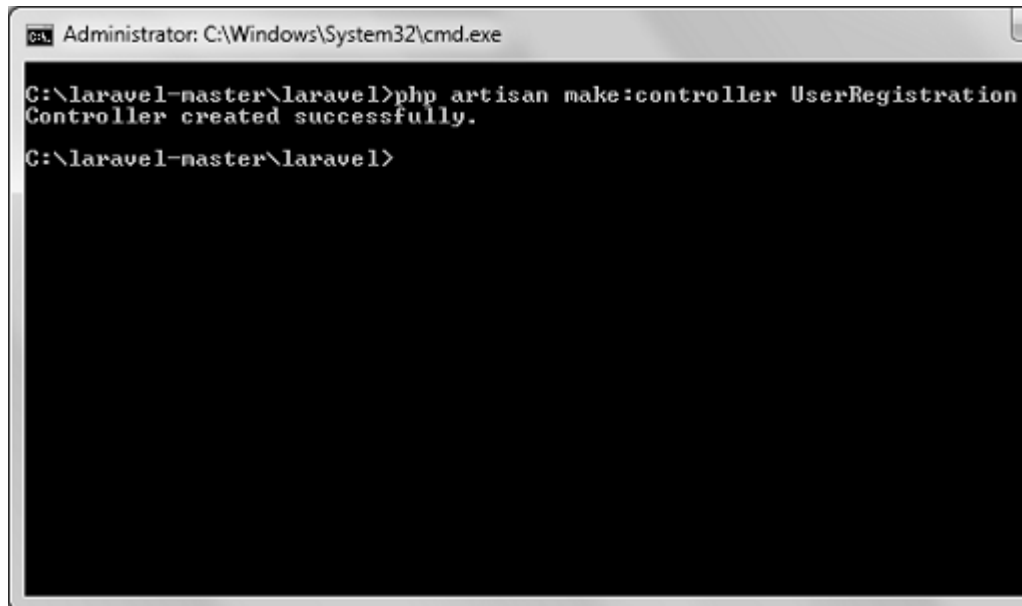
    </form>
```

```
</body>
</html>
```

Step 2 – Execute the below command to create a **UserRegistration** controller.

```
php artisan make:controller UserRegistration
```

Step 3 – After successful execution, you will receive the following output –

A screenshot of a Windows command prompt window titled "Administrator: C:\Windows\System32\cmd.exe". The prompt shows the command "C:\laravel-master\laravel>php artisan make:controller UserRegistration" being entered, followed by the output "Controller created successfully." and the prompt "C:\laravel-master\laravel>" on the next line.

```
Administrator: C:\Windows\System32\cmd.exe
C:\laravel-master\laravel>php artisan make:controller UserRegistration
Controller created successfully.
C:\laravel-master\laravel>
```

Step 4 – Copy the following code in **app/Http/Controllers/UserRegistration.php** controller.

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
use App\Http\Controllers\Controller;

class UserRegistration extends Controller {
    public function postRegister(Request $request){
        //Retrieve the name input field
        $name = $request->input('name');
        echo 'Name: '.$name;
        echo '<br>';

        //Retrieve the username input field
        $username = $request->username;
        echo 'Username: '.$username;
```



```

        echo '<br>';

        //Retrieve the password input field
        $password = $request->password;
        echo 'Password: '.$password;
    }
}

```

Step 5 – Add the following line in **app/Http/routes.php** file.

app/Http/routes.php

```

Route::get('/register',function(){
    return view('register');
});
Route::post('/user/register',array('uses'=>'UserRegistration@postRegister'));

```

Step 6 – Visit the following URL and you will see the registration form as shown in the below figure. Type the registration details and click Register and you will see on the second page that we have retrieved and displayed the user registration details.

http://localhost:8000/register

The output will look something like as shown in below the following images.

9. Display hello world in laravel

Step 1 – Add the following code to **app/Http/routes.php** file.

app/Http/routes.php

```

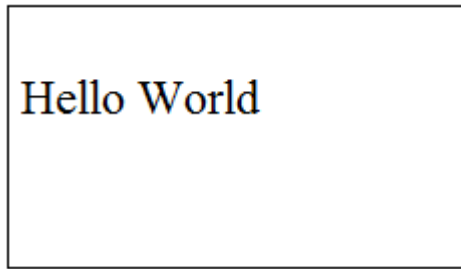
Route::get('/basic_response', function () {
    return 'Hello World';
});

```

Step 2 – Visit the following URL to test the basic response.

http://localhost:8000/basic_response

Step 3 – The output will appear as shown in the following image.



10. Display hello using header

Step 1 – Add the following code to **app/Http/routes.php** file.

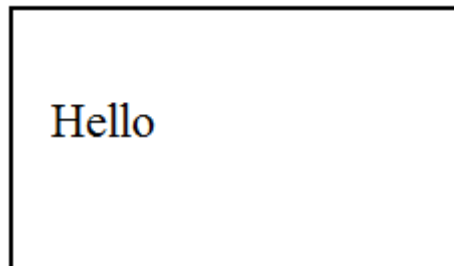
app/Http/routes.php

```
Route::get('/header',function(){  
    return response("Hello", 200)->header('Content-Type', 'text/html');  
});
```

Step 2 – Visit the following URL to test the basic response.

http://localhost:8000/header

Step 3 – The output will appear as shown in the following image.



11. Attaching cookie to laravel

Step 1 – Add the following code to **app/Http/routes.php** file.

```
Route::get('/cookie',function(){  
    return response("Hello", 200)->header('Content-Type', 'text/html')  
        ->withcookie('name','Virat Gandhi');  
});
```

Step 2 – Visit the following URL to test the basic response.

http://localhost:8000/cookie

Step 3 – The output will appear as shown in the following image.



12. Write a laravel routes to display output in format

Step 1 – Add the following line in **app/Http/routes.php** file.

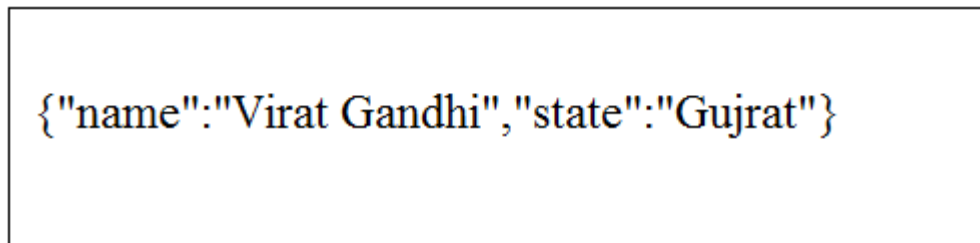
app/Http/routes.php

```
Route::get('json',function(){
    return response()->json(['name' => 'Virat Gandhi', 'state' => 'Gujarat']);
});
```

Step 2 – Visit the following URL to test the json response.

http://localhost:8000/json

Step 3 – The output will appear as shown in the following image.



13. Write a laravel code display php file in browser.

Step 1 – Copy the following code and save it at **resources/views/test.php**

```
<html>
  <body>
    <h1>Hello, World</h1>
  </body>
</html>
```

Step 2 – Add the following line in **app/Http/routes.php** file to set the route for the above view.

app/Http/routes.php

```
Route::get('/test', function(){
    return view('test');
});
```

Step 3 – Visit the following URL to see the output of the view.

http://localhost:8000/test

Step 4 – The output will appear as shown in the following image.



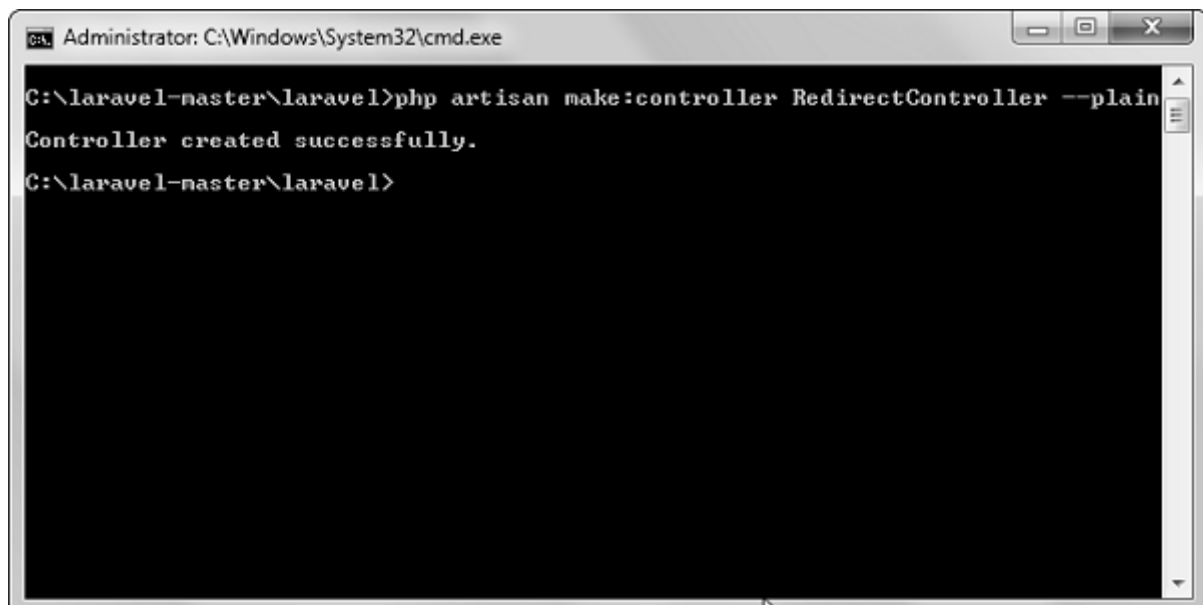
Hello, World

14. Create controller and access function of it in laravel

Step 1 – Execute the below command to create a controller called **RedirectController**.

```
php artisan make:controller RedirectController --plain
```

Step 2 – After successful execution, you will receive the following output –



Step 3 – Copy the following code to file **app/Http/Controllers/RedirectController.php**.

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Http\Requests;
use App\Http\Controllers\Controller;
```

```
class RedirectController extends Controller {
    public function index(){
        echo "Redirecting to controller's action.";
    }
}
```

Step 4 – Add the following lines in **app/Http/routes.php**.

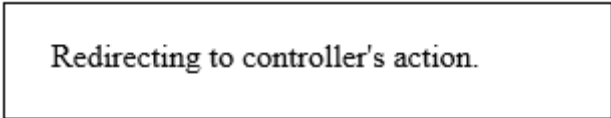
app/Http/routes.php

```
Route::get('rr', 'RedirectController@index');
Route::get('/redirectcontroller', function(){
    return redirect()->action('RedirectController@index');
});
```

Step 5 – Visit the following URL to test the example.

http://localhost:8000/redirectcontroller

Step 6 – The output will appear as shown in the following image.



Redirecting to controller's action.

15. Write Laravel Code For Addition, subtraction, Multiplication, division with Controller?

---create Controller using command
 php artisan make:controller calculatorcontroller
 =>Routes.php

Add this line to find Controller Method

```
//add
Route::get('/addtion','calculatorcontroller@add');

//sub   Route::get('/subtraction','calculatorcontroller@sub');

//mul
Route::get('/multiplication','calculatorcontroller@mul');

//div   Route::get('/division','calculatorcontroller@div');
```

 Controller
 -->App\Http\Controllers\ calculatorcontroller.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

use App\Http\Requests;

class calculatorcontroller extends Controller
{
    function Add()
```

```

        {
            $a=12;
            $b=13;
            $ans=$a+$b;
            echo 'addtion'.$ans;
        }
        function sub()
        {
            $x=25;
            $y=20;
            $z=$x-$y;
            return "subtraction"'.$z;
        }
        function mul()
        {
            $p=25;
            $q=20;
            $r=$p*$q;
            return "multiplication"'.$r;
        }
        function div()
        {
            $one=25;
            $two=20;
            $three=$p*$q;
            return "division"'.$three;
        }
    }
}

```

16. Laravel Route Program using Routes.php and Controller or print hello world using controller in laravel

```

===Routes.php
Route::get('/helloworld',function()
{
    return "helloworld";
});

```

```

==Controller.php
-----Create Controller using command

```

```

php artisan make:controller ExampleController

```

```

-----Routes.php
---add this line in Routes.php to finding the controller Method

```

```

Route::get('/hello','ExampleController@HelloWord');

```

```

==ExampleController
<?php

```

```

namespace App\Http\Controllers;

```

```

use Illuminate\Http\Request;

```

```

use App\Http\Requests;

```

```

class ExampleController extends Controller
{
    function HelloWord()
    {
        return "HelloWord using Controller";
    }
}

```

}

17. Example of route parameter with controller

1)write laravel code for routes Parameter in Routes.php and Controller.php

=====>Routes.php

comment //{---} this paramert pass in url like student/12

```
Route::get('student/{id}',function($id){
    echo "Your Id is student==>". $id;
});
```

=====>Controller

--create Controller using cmd

```
php artisan make:contoller ExampleController
```

--Routes.php

add this line to find controller method

comment //{---} this paramert pass in url like std/ABC

```
Route::get('/Std/{sname}','ExampleController@StudentName');
```

ExampleController.php

```
<?php
```

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

use App\Http\Requests;

class ExampleController extends Controller
{
    function HelloWorld()
    {
        return "HelloWord using Controller";
    }

    //this Method is call

    function StudentName($name)
    {
        return 'you name is==>'. $name;
    }
}
```

2)write laravel code for routes Parameter in Routes.php and Controller.php With Regulaer Exapression

==>Routes.php

```
Route::get('student/{sid}',function($id){
    echo "Your Id is student==>". $id;

    })->where('sid','[0-9]+');
```

```

=====
==>Controller.php
    --but only add where Condition in Routes.php file Controller as it is.

    Route::get('/Std/{sname}','ExampleController@StudentName')->where('sname','[A-Za-z]+');

=====OptionParameter=====
=====

```

3)write laravel code for routes Parameter in Routes.php and Controller.php With OptionalParameter

```

Route::get('student/{id?}',function($id=null){

    echo "Your Id is student==>".$id;

});
Route::get('student/{name?}',function($name='Hello'){

    echo "Your Id is student==>".$name;

});

```

18. Examples related to route file , blade file and controller file.

1)Write Laravel code FOR Pass the Route from blade file in Routes.php and Controller:

```

==>Routes.php
    pass/student this url pass to student this anything but same as both

    Route::get('pass/student',function()
    {
        return view('student');
    });

```

==>Controller

=>Routes.php
Add this line to find method

```
Route::get('/view','ExampleController@view');
```

=>Examplecontroller.php
<?php

```

namespace App\Http\Controllers;

use Illuminate\Http\Request;

use App\Http\Requests;

class ExampleController extends Controller
{
    function HelloWorld()
    {
        return "HelloWord using Controller";
    }
    function StudentName($name)
    {
        return 'you name is==>'.$name;
    }
    //call this methods now
    function view()
    {

```



```

        return view('student');
    }
}

```

2)Write Laravel code FOR Pass the Route form bladefile in Routes.php using Controller with values ?

==>Routes.php

```

Route::get('value/{id}',function($id)
{
    return view('student',compact('id'));
});

```

==>student.blade.php

ok student{{ \$id }}

==>Controller

--- Routes.php

```

Route::get('/view/{name}','ExampleController@view');

```

==>Controller.php

```

function view($name)
{
    return view('student',compact('name'));
}

```

===>std.blade.php //this blade file save in view Folder

name is you{{ \$name }}

19. Write Laravel code two input(text)box and one button when button is press and all the textbox value is pass in controller method using post Method???

===create Controller using command

```
php artisan make:controller ExampleController
```

=>Routes.php

Add this line to find Controller Method

```

Route::get('/getform','ExampleController@getform');
Route::post('/postdata','ExampleController@postform');

```

===>Controllere

ExampleController.php

```
<?php
```

```

namespace App\Http\Controllers;

```

```

use Illuminate\Http\Request;

```

```

use App\Http\Requests;

```

```

class ExampleController extends Controller
{
    function HelloWorld()
    {
        return "HelloWord using Controller";
    }

    //Call this Methods

    function getform()
    {
        return view('form');
    }
    function postform(Request $request)
    {
        $name=$request->get('name');
        $mobile=$request->get('mobile');
        $password=$request->input('password');
        $address=$request->input('address');

        return 'name=>'.$name.'-'. 'password=>'.$password.'-'. 'mobile=>'. $mobile.'-'. '$address=>'. $address;

        //or
        echo $name;
        echo $mobile;
        echo $password;
        echo $address;
    }
}

```

=====Form.blade.php=====

```

<!DOCTYPE html>
<html>
<head>
    <title>This Student INfo Form</title>
</head>
<body>
    <form action="/postdata" method="post">
        {{ csrf_field() }}
        <input type="text" name="name">
        <input type="text" name="password">
        <input type="text" name="mobile">
        <input type="text" name="address">
        <input type="submit" name="ok" value="Click">

        <input type="Reset" name="old" value="Reset">

    </form>
</body>
</html>

```

=>when this error is Come

TokenMismatchException in VerifyCsrfToken.php line 67:

add one line after form tag like

```
<form action="/postdata" method="post">
```

```
{{ csrf_field() }}
```

oR--

```
<input type="hidden" name="_token" value="{{ csrf_token() }}">
```

❖ Example of Custom Command

1)To create a new console command named

```
---php artisan make:console Student --command=class:std
    -make:command
    -make:console.
```

it creates student.php in app/console/commands/student.php

2)command is not registered so it must be Register

```
--/app/console/Kernel.php

protected $commands = [

    'App\Console\Commands\Student'

];
```

3)php artisan class:std

4)The name and signature of the console command.

-we write the signature is same as the command name so execute this

```
protected $signature = 'class:std';
```

-The console command description.

```
protected $description = 'Command description';
```

5)when command is write in cmd than call the handle();

```
public function handle()
{
    echo "my Custome Command";

    $this->line("Some text");
    $this->info("Hey, watch this !");
    $this->comment("Just a comment passing by");
    $this->question("Why did you do that?");
    $this->error("Ops, that should not happen.");
}
```

Chapter – 4.1 [Blade Template]

INTRODUCTION

Blade is the simple, yet powerful templating engine provided with Laravel. Unlike other popular PHP templating engines, Blade does not restrict you from using plain PHP code in your views. All Blade views are compiled into plain PHP code and cached until they are modified, meaning Blade adds essentially zero overhead to your application. Blade view files use the `.blade.php` file extension and are typically stored in the `resources/views` directory.

☺ Template Inheritance:

☺ Master Layout:

DEFINING A LAYOUT

Two of the primary benefits of using Blade are *template inheritance* and *sections*. To get started, let's take a look at a simple example. First, we will examine a "master" page layout. Since most web applications maintain the same general layout across various pages, it's convenient to define this layout as a single Blade view:

```
<!-- Stored in resources/views/layouts/master.blade.php -->

<html>
  <head>
    <title>App Name -
    @yield('title')</title> </head>
  <body>
    @section('sidebar')
      This is the master
      sidebar. @show

    <div class="container">
      @yield('content')
    </div>
  </body>
</html>
```

As you can see, this file contains typical HTML mark-up. However, take note of the `@section` and `@yield` directives. The `@section` directive, as the name implies, defines a section of content, while the `@yield` directive is used to display the contents of a given section.

Now that we have defined a layout for our application, let's define a child page that inherits the layout.

☺ Extending the master layout:

EXTENDING A LAYOUT

When defining a child page, you may use the Blade `@extends` directive to specify which layout the child page should "inherit". Views which `@extends` a Blade layout may inject content into the layout's sections using `@section` directives. Remember, as seen in the example above, the contents of these sections will be displayed in the layout using `@yield`:

```
<!-- Stored in resources/views/child.blade.php -->

@extends('layouts.master')

@section('title', 'Page Title')

@section('sidebar')
    @parent

    <p>This is appended to the master
    sidebar.</p> @endsection

@section('content')
    <p>This is my body
    content.</p> @endsection
```

In this example, the `sidebar` section is utilizing the `@parent` directive to append (rather than overwriting) content to the layout's sidebar. The `@parent` directive will be replaced by the content of the layout when the view is rendered.

Of course, just like plain PHP views, Blade views may be returned from routes using the global `view` helper function:

```
Route::get('blade', function () {
    return view('child');
});
```

☺ Displaying variables:

You may display data passed to your Blade views by wrapping the variable in "curly" braces. For example, given the following route:

```
Route::get('greeting', function () {
    return view('welcome', ['name' => 'Samantha']);
});
```

You may display the contents of the `name` variable like so:

```
Hello, {{ $name }}.
```

Of course, you are not limited to displaying the contents of the variables passed to the view. You may also echo the results of any PHP function. In fact, you can put any PHP code you wish inside of a Blade echo statement:

```
The current UNIX timestamp is {{ time() }}.
```

Note: Blade `{ }` statements are automatically sent through PHP's `htmlspecialchars` function to prevent XSS attacks.

Blade & JavaScript Frameworks

Since many JavaScript frameworks also use "curly" braces to indicate a given expression should be displayed in the browser, you may use the `@` symbol to inform the Blade rendering engine an expression should remain untouched. For example:

```
<h1>Laravel</h1>

Hello, @{{ name }}.
```

In this example, the `@` symbol will be removed by Blade; however, `{{ name }}` expression will remain untouched by the Blade engine, allowing it to instead be rendered by your JavaScript framework.

Echoing Data If It Exists

Sometimes you may wish to echo a variable, but you aren't sure if the variable has been set. We can express this in verbose PHP code like so:

```
{{ isset($name) ? $name : 'Default' }}
```

However, instead of writing a ternary statement, Blade provides you with the following convenient short-cut:

```
{{ $name or 'Default' }}
```

In this example, if the `$name` variable exists, its value will be displayed. However, if it does not exist, the word `Default` will be displayed.

Displaying Unescaped Data

By default, Blade `{{ }}` statements are automatically sent through PHP's `htmlspecialchars` function to prevent XSS attacks. If you do not want your data to be escaped, you may use the following syntax:

```
Hello, {!! $name !!}.
```

Note: Be very careful when echoing content that is supplied by users of your application. Always use the double curly brace syntax to escape any HTML entities in the content.

☺ Blade conditional statements:

In addition to template inheritance and displaying data, Blade also provides convenient short-cuts for common PHP control structures, such as conditional statements and loops. These short-cuts provide a very clean, terse way of working with PHP control structures, while also remaining familiar to their PHP counterparts.

If Statements

You may construct if statements using the `@if`, `@elseif`, `@else`, and `@endif` directives. These directives function identically to their PHP counterparts:

```
@if (count($records) === 1)
    I have one record!
@elseif (count($records) > 1)
    I have multiple records!
@else
    I don't have any
records! @endif
```

For convenience, Blade also provides an `@unless` directive:

```
@unless (Auth::check())
    You are not signed in.
@endunless
```

☺ Blade Loops:

In addition to conditional statements, Blade provides simple directives for working with PHP's supported loop structures. Again, each of these directives functions identically to their PHP counterparts:

```
@for ($i = 0; $i < 10; $i++)
    The current value is {{ $i }}
@endfor

@foreach ($users as $user)
    <p>This is user {{ $user->id }}</p> @endforeach

@forelse ($users as $user)
    <li>{{ $user->name }}</li>
@empty
    <p>No users</p>
@endforelse

@while (true)
    <p>I'm looping forever.</p>
@endwhile
```

Including Sub-Views

Blade's `@include` directive, allows you to easily include a Blade view from within an existing view. All variables that are available to the parent view will be made available to the included view:

```
<div>
    @include('shared.errors')

    <form>
        <!-- Form Contents -->
    </form>
</div>
```

Even though the included view will inherit all data available in the parent view, you may also pass an array of extra data to the included view:

```
@include('view.name', ['some' => 'data'])
```

Note: You should avoid using the `__DIR__` and `__FILE__` constants in your Blade views, since they will refer to the location of the cached view.

Rendering Views For Collections

You may combine loops and includes into one line with Blade's `@each` directive:

```
@each('view.name', $jobs, 'job')
```

The first argument is the view partial to render for each element in the array or collection. The second argument is the array or collection you wish to iterate over, while the third argument is the variable name that will be assigned to the current iteration within the view. So, for example, if you are iterating over an array of `jobs`, typically you will want to access each job as a `job` variable within your view partial.

You may also pass a fourth argument to the `@each` directive. This argument determines the view that will be rendered if the given array is empty.

```
@each('view.name', $jobs, 'job', 'view.empty')
```

Comments

Blade also allows you to define comments in your views. However, unlike HTML comments, Blade comments are not included in the HTML returned by your application:

```
{{-- This comment will not be present in the rendered HTML --}}
```


☺ Executing PHP functions in blade:

SERVICE INJECTION

The `@inject` directive may be used to retrieve a service from the Laravel service container. The first argument passed to `@inject` is the name of the variable the service will be placed into, while the second argument is the class / interface name of the service you wish to resolve:

```
@inject('metrics', 'App\Services\MetricsService')

<div>
    Monthly Revenue: {{ $metrics->monthlyRevenue() }}
</div>
```

EXTENDING BLADE

Blade even allows you to define your own custom directives. You can use the `directive` method to register a directive. When the Blade compiler encounters the directive, it calls the provided callback with its parameter.

The following example creates a `@datetime($var)` directive which formats a given `$var`:

```
<?php

namespace App\Providers;

use Blade;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Perform post-registration booting of services.
     *
     * @return void
     */
    public function boot()
    {
        Blade::directive('datetime', function($expression) {
            return "<?php echo with{$expression}->format('m/d/Y
H:i'); ?>";
        });
    }

    /**
     * Register bindings in the container.
     *
     * @return void
     */
    public function register()
    {
    }
}
```

```

        //
    }
}

```

As you can see, Laravel's `with` helper function was used in this directive. The `with` helper simply returns the object / value it is given, allowing for convenient method chaining. The final PHP generated by this directive will be:

```
<?php echo with($var)->format('m/d/Y H:i'); ?>
```

❖ Master page or blade template related programs –

1. Write a program to create master page and access other pages from it.

Step 1 – Create a master template and save it at **resources/views/layouts/master.blade.php**.

```

<html>

<head>
    <title>@yield('title')</title>
</head>

<body>
    @section('sidebar')
        This is the master sidebar.
    @show

    <div class = "container">
        @yield('content')
    </div>

</body>
</html>

```

Step 2 – Now, create another page and extend the master template and save it at **resources/views/page.blade.php**

```

@extends('layouts.master')
@section('title', 'Page Title')

@section('sidebar')

```

```

@parent

<p>This is appended to the master sidebar.</p>

@endsection

@section('content')

<h2>{{ $name }}</h2>

<p>This is my body content.</p>

@endsection

```

Step 3 – Now, set up the route to view this template. Add the following line at **app/Http/routes.php**

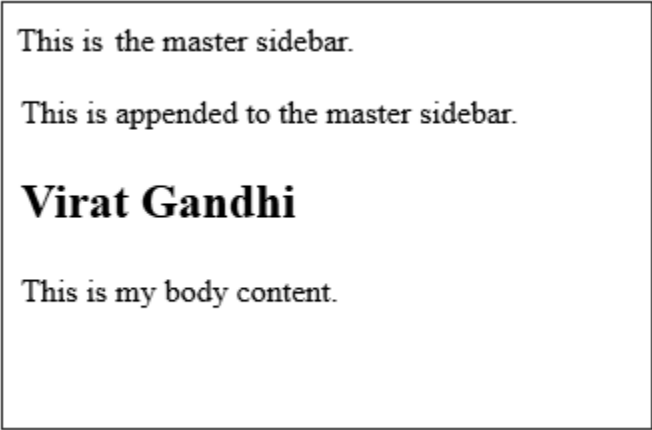
```

Route::get('blade', function () {
    return view('page', array('name' => 'Virat Gandhi'));
});

```

Step 4 – Visit the following URL to view the blade template example.

http://localhost:8000/blade



The screenshot shows the output of the blade template. It consists of a rectangular box containing the following text: "This is the master sidebar." followed by "This is appended to the master sidebar." on the next line. Below these is a large heading "Virat Gandhi" in bold, and at the bottom, "This is my body content.".

2. Write a program which demonstrate concept of master page

1) app/Http/routes.php

```

Route::get('/', function () {
    return view('home');
})->name("home");

```

```

Route::get('/about', function () {
    return view('about');
})->name("about");

```

```

Route::get('/contact', function () {
    return view('contact');
})->name("contact");

```

```

Route::get('/feedback', function () {
    return view('feedback');
})->name("feedback");

```

```

Route::get('/blog', function () {
    return view('blog');
})->name("blog");

```

```
Route::get('/login', function () {
    return view('login');
})->name("login");
```

```
Route::get('/drop', function () {
    return view('drop');
})->name("drop");
```

2) resources/views/home.blade.php

```
@extends('layouts.master')
@section('content')
<h1>Hi This is My First Php Blade File</h1>
```

3) resources/views/about.blade.php

```
@extends('layouts.master')
@section('content')
<h1>This is About Page</h1>
@stop
```

4) resources/views/contact.blade.php

```
@extends('layouts.master')
@section('content')
<h1>This is Contact Page</h1>
@stop
```

5) resources/views/feedback.blade.php

```
@extends('layouts.master')
@section('content')
<h1>This is FEED ACK PAGE</h1>
@stop
```

6) resources/views/blog.blade.php

```
@extends('layouts.master')
@section('title')
    This is Blog Page
@stop
@section('content')
<h1>This is Blog Page</h1>
@stop
```

7) resources/views/login.blade.php

```
@extends('layouts.master')
@section('content')
<div class="container">
<h2>Vertical (basic) form</h2>
{!!Form::open(['url'=>'', 'id'=>'form'])!!}
<div class="form-group">
<label for="email">Email:</label>
{!!Form::text('first_name',null,['id'=>'first_name','class'=>'form-control',
'placeholder'=>'Enter email'])!!}
</div>
<div class="form-group">
<label for="email">Email:</label>
{!!Form::text('first_name',null,['id'=>'first_name','class'=>'form-control',
'placeholder'=>'Enter email'])!!}
</div>
<div class="checkbox">
<label><input type="checkbox"> Remember me</label>
</div>
<button type="submit" class="btn btn-default">Submit</button>
{!!Form::close()!!}
</div>
@stop
```

8) resources/views/drop.blade.php

```
@extends('layouts.master')
@section('content')
<div class="dropdown">
```

```

        <button class="btn btn-primary dropdown-toggle" type="button" data-
toggle="dropdown">Dropdown Example
        <span class="caret"></span></button>
        <ul class="dropdown-menu">
        <li><a href="#">html</a></li>
        <li><a href="#">css</a></li>
        <li><a href="#">Javascript</a></li>
        </div>
        @stop

```

9) resources/view/includes/header.blade.php

```

<nav class="navbar navbar-inverse">
    <div class="container-fluid">
        <div class="navbar-header">
            <a class="navbar-brand" href="#">WebSiteName</a>
        </div>
        <ul class="nav navbar-nav">
            <li class="active"><a href="{{ route('home') }}">Home</a></li>
            <li><a href="{{ route('contact') }}">Contact</a></li>
            <li><a href="{{ route('about') }}">About</a></li>
            <li><a href="{{ route('feedback') }}">Feedback</a></li>
            <li><a href="{{ route('blog') }}">Blog</a></li>
            <li><a href="{{ route('login') }}">login</a></li>
            <li><a href="{{ route('drop') }}">drop</a></li>
        </ul>
        <ul class="nav navbar-nav navbar-right">
            <li><a href="#"><span class="glyphicon glyphicon-user"></span> Sign Up</a></li>
            <li><a href="#"><span class="glyphicon glyphicon-log-in"></span> Login</a></li>
        </ul>
    </div>
</nav>

```

10)resources/view/includes/footer.blade.php

```

<nav class="navbar navbar-inverse navbar-fixed-bottom">
    <div class="container-fluid">
        <div class="navbar-header">
            <a class="navbar-brand" href="#">WebSiteName</a>
        </div>
        <ul class="nav navbar-nav">
            <li class="active"><a href="#">Home</a></li>
            <li><a href="#">Page 1</a></li>
            <li><a href="#">Page 2</a></li>
            <li><a href="#">Page 3</a></li>
        </ul>
    </div>
</nav>

```

11)resources/view/layouts/master.blade.php

```

<!DOCTYPE html>
<html lang="en">
<head>
    <title>Bootstrap Example</title>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=5">
    <link rel="stylesheet" href="{{ URL::asset('assets/css/bootstrap.min.css') }}">
    <script src="{{ URL::asset('assets/js/jquery.min.js') }}"></script>
    <script src="{{ URL::asset('assets/js/bootstrap.min.js') }}"></script>

</head>
<body>

```

```

<div class="container">

```

```

        @include('includes.header')

```

```

        @yield('content')

```

```

        @include('includes.footer')

    </div>

</body>
</html>

```

3. Write a program to create master page and child page

1) app/Http/routes.php

```

Route::get('/', function () {
    return view('child');
})->name("chile");

```

2)Layouts/Master.blade.php

```

<!DOCTYPE html>
<html>
<head>

    <title>AppName:@yield('title')</title>
    <style type="text/css">
        .container{
            background-color: red;
        }
        #id
        {
            background-color: pink;
        }
        .head
        {
            background-color: green;
            margin: left 12px;
            width: 500px;
            height: 50px
        }
    </style>
</head>
<body>
<div class="head">
    @yield('header')
</div>

    @section('siderbar')
    @show

    <div class="container">
        @yield('content')
    </div>
    <div id="id">
        @yield('footer')
    </div>
</body>
</html>

```

3)Child.blade.php

```

@extends('layouts.master')
@section('title','My Page')
<br>

@section('header')
<br>
<a href="">Home</a>| <a href="">About</a>| <a href="">conact</a>| <a href="">Login</a>
@endsection

```

```

@section('siderbar')
<p>@parent
this is sidebar page
</p>
@endsection

@section('content')
<h1>
Wellcome to All IN My Blade Page</h1>
@endsection

@section('footer')
By..Vip in 2017
@endsection

```

4. Write a program which display example of condition and loops

1) app/Http/routes.php

Route.php

```

Route::get('name', function () {
return view('student', ['name' => 'Samantha']);
});
Route::get(Foreach, function () {
return view(Foreach);
});

Route::get(Forloop, function () {
return view(Forloop);
});

Route::get(IF_ELSE, function () {
return view('IF_ELSE');
});

```

resources/views/

2)student.blade.php

```

student name is
{{$name}}
<br>
{{ isset($name) ? $name : 'Default' }}
<br>
Hello=>hru==>{!! $name !!}.
=====ForLoop=====

```

3)Foreach.blade.php

```

<?php
$a=array("Name"=>"Peter",'vip',"Age"=>"41",'12',"Country"=>"USA",'India');
//print_r(array_values($a));

?>
@foreach ($a as $user)

<p>This is user {{ $user}}</p>

@endforeach

```

4)Forloop.blade.php

```
@for ($i = 1; $i <= 10; $i++)
    <br>
    The current value is {{ $i }}
```

```
@endfor
```

5)IF_ELSE.blade.php

```
<?php $nm = '12' ?>
@if (count($name) === 1) I have one record!
@endif
<br>
@if ($nm > 1) I have multiple records!
@else
<br>
I don't have any records!
@endif
```

5)TimeShow.blade

```
<center>
<b>Time is Now</b><?php echo (date('m/d/Y H:i:s')) ; ?>
</center>
```

5. Write a program which display login form using blade.**1)Routes.php**

```
Route::get('/login',function(){
    return view('login');
});
Route::post('/condtion','DeomController@condtion');
```

2)DeomController.php

```
<?php

namespace App\Http\Controllers;

use App\Http\Requests;
use Illuminate\Http\Request;

classDeomControllerextends Controller
{
functioncondtion(Request $res)
{
    $name=$res->get('name');
    //echo $name;
    $pass=$res->get('pass');
    //echo $pass;
    return view('condtion',compact('name','pass'));
}
}
```

3)Layouts/Master.blade.php

```
<!DOCTYPE html>
<html>
<head>

    <title>AppName:@yield('title')</title>
    <style type="text/css">
        .container{
```



```

        background-color: red;
    }
    #id
    {
        background-color: pink;
    }
    .head
    {
        background-color: green;
        margin: left 12px;
        width: 500px;
        height: 50px
    }
</style>
</head>
<body>
<div class="head">
    @yield('header')
</div>

    <div class="container">
        @yield('content')
    </div>
    <div id="id">
        @yield('footer')
    </div>
</body>
</html>

```

4)login.blade.php

```

@extends('layouts.master')
@section('title','LoginPage')
@section('header')
<br>
<center>
<a href="/index">Home</a>|<a href="/about">About</a>|<a href="/contact">conact</a>|<a
href="/login">Login</a>
</center>
@endsection
@section('content')
<div class="form">
<form action="condition" method="post">
<input type="hidden" name="_token" value="{{ csrf_token() }}">
<table align="center">
<tr><td>
        EnterName:<td>
        <input type="text" name="name" ><br>
        <tr><td>Pasword<td>
        <input type="password" name="pass">
        <br>
        <input type="submit" name=""></td></tr>
</table>
</form>
</div>
@endsection

@section('footer')
<br>
By..Dev in 2017
@endsection

```

5)Condtion.blade.php

```

<!DOCTYPE html>
<html>
<head>
    <title>Wellcome to Profile</title>
    <style type="text/css">
        h3{
            color: red;
        }
        h2{
            color: green;
        }
        .Profile
        {
            margin-left:100px;
        }
    </style>
</head>
<body>

<div class="Profile">
    <br>
    UserName:{{$name}}
    <br>
    PassWord{{$pass}}
</div>
<div class="validtion">
    @if($name=="")
    <h3>
    Please Enter Ename</h3>
    @endif
    @if($pass=="")
    <h3>
    Please Enter Password
    </h3>
    @endif
</div>
<div class="condtion">

    @if ($name == $pass)
    <h2>
    UserName and PassWord are Same!</h2>

    @elseif(strlen($pass) < 6)
    <h3>
    Please Enter PassWordMorethan six character!    </h3>
    @else
    <h1>
    <marquee>
    wellcome to Profilepage</marquee></h1>
    @endif
</div>

</body>
</html>

```

Chapter – 4.2 [SQL Interaction]

☺ Introduction:

Laravel makes connecting with databases and running queries extremely simple across a variety of database back-ends using either raw SQL, the fluent query builder, and the Eloquent ORM. Currently, Laravel supports four database systems:

- MySQL
- Postgres
- SQLite
- SQL Server

☺ Running RAW SQL Queries:

Once you have configured your database connection, you may run queries using the `DB` facade. The `DB` facade provides methods for each type of query: `select`, `update`, `insert`, `delete`, and `statement`.

Running A Select Query

To run a basic query, we can use the `select` method on the `DB` facade:

```
<?php

namespace App\Http\Controllers;

use DB;
use App\Http\Controllers\Controller;

class UserController extends Controller
{
    /**
     * Show a list of all of the application's users.
     *
     * @return Response
     */
    public function index()
    {
        $users = DB::select('select * from users where active = ?',
[1]);

        return view('user.index', ['users' => $users]);
    }
}
```

The first argument passed to the `select` method is the raw SQL query, while the second argument is any parameter bindings that need to be bound to the query. Typically, these are

the values of the `where` clause constraints. Parameter binding provides protection against SQL injection.

The `select` method will always return an `array` of results. Each result within the array will be a PHP `stdClass` object, allowing you to access the values of the results:

```
foreach ($users as $user) {  
    echo $user->name;  
}
```

Using Named Bindings

Instead of using `?` to represent your parameter bindings, you may execute a query using named bindings:

```
$results = DB::select('select * from users where id = :id', ['id' =>  
1]);
```

Running An Insert Statement

To execute an `insert` statement, you may use the `insert` method on the `DB` facade. Like `select`, this method takes the raw SQL query as its first argument, and bindings as the second argument:

```
DB::insert('insert into users (id, name) values (?, ?)',  
[1, 'Dayle']);
```

Running An Update Statement

The `update` method should be used to update existing records in the database. The number of rows affected by the statement will be returned by the method:

```
$affected = DB::update('update users set votes = 100 where name =  
?', ['John']);
```

Running A Delete Statement

The `delete` method should be used to delete records from the database. Like `update`, the number of rows deleted will be returned:

```
$deleted = DB::delete('delete from users');
```

Running A General Statement

Some database statements should not return any value. For these types of operations, you may use the `statement` method on the `DB` facade:

```
DB::statement('drop table users');
```

LISTENING FOR QUERY EVENTS

If you would like to receive each SQL query executed by your application, you may use the `listen` method. This method is useful for logging queries or debugging. You may register your query listener in a service provider:

```
<?php

namespace App\Providers;

use DB;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        DB::listen(function($sql, $bindings, $time) {
            //
        });
    }

    /**
     * Register the service provider.
     *
     * @return void
     */
    public function register()
    {
        //
    }
}
```

😊 Database Transactions:

To run a set of operations within a database transaction, you may use the `transaction` method on the `DB` facade. If an exception is thrown within the transaction `Closure`, the transaction will automatically be rolled back. If the `Closure` executes successfully, the transaction will automatically be committed. You don't need to worry about manually rolling back or committing while using the `transaction` method:

```
DB::transaction(function () {
    DB::table('users')->update(['votes' => 1]);

    DB::table('posts')->delete();
});
```

Manually Using Transactions

If you would like to begin a transaction manually and have complete control over rollbacks and commits, you may use the `beginTransaction` method on the `DB` facade:

```
DB::beginTransaction();
```

You can rollback the transaction via the `rollBack` method:

```
DB::rollBack();
```

Lastly, you can commit a transaction via the `commit` method:

```
DB::commit();
```

Note: Using the `DB` facade's transaction methods also controls transactions for the [query builder](#) and [Eloquent ORM](#).

USING MULTIPLE DATABASE CONNECTIONS

`--migration`

When using multiple connections, you may access each connection via the `connection` method on the `DB` facade. The `name` passed to the `connection` method should correspond to one of the connections listed in your configuration file:

`config/database.php`

```
$users = DB::connection('foo')->select(...);
```

You may also access the raw, underlying PDO instance using the `getPdo` method on a connection instance:

```
$pdo = DB::connection()->getPdo();
```

Chapter – 5.1 [Eloquent ORM]

☺ Eloquent ORM Models:

☺ Naming conventions:

To get started, let's create an Eloquent model. Models typically live in the `app` directory, but you are free to place them anywhere that can be auto-loaded according to your `composer.json` file. All Eloquent models extend `Illuminate\Database\Eloquent\Model` class.

The easiest way to create a model instance is using the `make:model` Artisan command:

```
php artisan make:model User
```

If you would like to generate a database migration when you generate the model, you may use the `--migration` or -m option:

```
php artisan make:model User --migration
```

```
php artisan make:model User -m
```

ELOQUENT MODEL NAMING CONVENTIONS

Now, let's look at an example `Flight` model class, which we will use to retrieve and store information from our `flights` database table:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    //
}
```

☺ Table name and primary keys:

Table Names

Note that we did not tell Eloquent which table to use for our `Flight` model. The "snake case", plural name of the class will be used as the table name unless another name is explicitly specified. So, in this case, Eloquent will assume the `Flight` model stores records in the `flights` table. You may specify a custom table by defining a `table` property on your model:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The table associated with the model.
     *
     * @var string
     */
    protected $table = 'my_flights';
}
```

Primary Keys

Eloquent will also assume that each table has a primary key column named `id`. You may

```
define a
    $primaryKey property to override this convention.
```

☺ Timestamps:

Timestamps

By default, Eloquent expects `created_at` and `updated_at` columns to exist on your tables. If you do not wish to have these columns automatically managed by Eloquent, set the `$timestamps` property on your model to `false`:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * Indicates if the model should be timestamped.
     *
     * @var bool
     */
    public $timestamps = false;
}
```

If you need to customize the format of your timestamps, set the `$dateFormat` property on your model. This property determines how date attributes are stored in the database, as well as their format when the model is serialized to an array or JSON:

```
<?php

namespace App;
```



```
use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The storage format of the model's date columns.
     *
     * @var string
     */
    protected $dateFormat = 'U';
}
```

☺ Basic Operations:

CREATE

To store posts data a table need to be created in the database. It is create operation.

RETRIEVING SINGLE MODELS / AGGREGATES

Of course, in addition to retrieving all of the records for a given table, you may also retrieve single records using `find` and `first`. Instead of returning a collection of models, these methods return a single model instance:

```
// Retrieve a model by its primary key...
$flight = App\Flight::find(1);

// Retrieve the first model matching the query constraints...
$flight = App\Flight::where('active', 1)->first();
```

Not Found Exceptions

Sometimes you may wish to throw an exception if a model is not found. This is particularly useful in routes or controllers. The `findOrFail` and `firstOrFail` methods will retrieve the first result of the query. However, if no result is found, a

`Illuminate\Database\Eloquent\ModelNotFoundException` will be thrown:

```
$model = App\Flight::findOrFail(1);

$model = App\Flight::where('legs', '>', 100)->firstOrFail();
```

If the exception is not caught, a 404 HTTP response is automatically sent back to the user, so it is not necessary to write explicit checks to return 404 responses when using these methods:

```
Route::get('/api/flights/{id}', function ($id) {
    return App\Flight::findOrFail($id);
});
```

RETRIEVING AGGREGATES

Of course, you may also use `count`, `sum`, `max`, and other aggregate functions provided by the query builder. These methods return the appropriate scalar value instead of a full model instance:

```
$count = App\Flight::where('active', 1)->count();
```

```
$max = App\Flight::where('active', 1)->max('price');
```

INSERTING & UPDATING MODELS

Basic Inserts

To create a new record in the database, simply create a new model instance, set attributes on the model, then call the `save` method:

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use App\Flight;
```

```
use Illuminate\Http\Request;
```

```
use App\Http\Controllers\Controller;
```

```
class FlightController extends Controller
{
    /**
     * Create a new flight instance.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        // Validate the request...

        $flight = new Flight;

        $flight->name = $request->name;

        $flight->save();
    }
}
```

In this example, we simply assign the `name` parameter from the incoming HTTP request to the `name` attribute of the `App\Flight` model instance. When we call the `save` method, a record will be inserted into the database. The `created_at` and `updated_at` timestamps will automatically be set when the `save` method is called, so there is no need to set them manually.

Basic Updates

The `save` method may also be used to update models that already exist in the database. To update a model, you should retrieve it, set any attributes you wish to update, and then call the `save` method. Again, the `updated_at` timestamp will automatically be updated, so there is no need to manually set its value:

```
$flight = App\Flight::find(1);

$flight->name = 'New Flight Name';

$flight->save();
```

Updates can also be performed against any number of models that match a given query. In this example, all flights that are `active` and have a `destination` of `San Diego` will be marked as `delayed`:

```
App\Flight::where('active', 1) -
    >where('destination', 'San Diego')
    ->update(['delayed' => 1]);
```

The `update` method expects an array of column and value pairs representing the columns that should be updated.

Mass Assignment

You may also use the `create` method to save a new model in a single line. The inserted model instance will be returned to you from the method. However, before doing so, you will need to specify either a `fillable` or `guarded` attribute on the model, as all Eloquent models protect against mass-assignment.

A mass-assignment vulnerability occurs when a user passes an unexpected HTTP parameter through a request, and that parameter changes a column in your database you did not expect. For example, a malicious user might send an `is_admin` parameter through an HTTP request, which is then mapped onto your model's `create` method, allowing the user to escalate themselves to an administrator.

So, to get started, you should define which model attributes you want to make mass assignable. You may do this using the `$fillable` property on the model. For example, let's make the `name` attribute of our `Flight` model mass assignable:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The attributes that are mass assignable.
     */
}
```

```
*
* @var
* array */
protected $fillable = ['name'];
}
```

Once we have made the attributes mass assignable, we can use the `create` method to insert a new record in the database. The `create` method returns the saved model instance:

```
$flight = App\Flight::create(['name' => 'Flight 10']);
```

While `$fillable` serves as a "white list" of attributes that should be mass assignable, you may also choose to use `$guarded`. The `$guarded` property should contain an array of attributes that you do not want to be mass assignable. All other attributes not in the array will be mass assignable. So, `$guarded` functions like a "black list". Of course, you should use either `$fillable` or `$guarded` - not both:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Flight extends Model
{
    /**
     * The attributes that aren't mass assignable.
     *
     * @var array
     */
    protected $guarded = ['price'];
}
```

In the example above, all attributes **except for price** will be mass assignable.

Other Creation Methods

There are two other methods you may use to create models by mass assigning attributes: `firstOrCreate` and `firstOrCreate`. The `firstOrCreate` method will attempt to locate a database record using the given column / value pairs. If the model can not be found in the database, a record will be inserted with the given attributes.

The `firstOrCreate` method, like `firstOrCreate` will attempt to locate a record in the database matching the given attributes. However, if a model is not found, a new model instance will be returned. Note that the model returned by `firstOrCreate` has not yet been persisted to the database. You will need to call `save` manually to persist it:

```
// Retrieve the flight by the attributes, or create it if it doesn't
exist...
$flight = App\Flight::firstOrCreate(['name' => 'Flight 10']);
```

```
// Retrieve the flight by the attributes, or instantiate a
new instance...
$flight = App\Flight::firstOrCreate(['name' => 'Flight 10']);
```

DELETING MODELS

To delete a model, call the `delete` method on a model instance:

```
$flight = App\Flight::find(1);

$flight->delete();
```

Deleting An Existing Model By Key

In the example above, we are retrieving the model from the database before calling the `delete` method. However, if you know the primary key of the model, you may delete the model without retrieving it. To do so, call the `destroy` method:

```
App\Flight::destroy(1);

App\Flight::destroy([1, 2, 3]);

App\Flight::destroy(1, 2, 3);
```

Deleting Models By Query

Of course, you may also run a delete query on a set of models. In this example, we will delete all flights that are marked as inactive:

```
$deletedRows = App\Flight::where('active', 0)->delete();
```

Soft Deleting

In addition to actually removing records from your database, Eloquent can also "soft delete" models. When models are soft deleted, they are not actually removed from your database. Instead, a `deleted_at` attribute is set on the model and inserted into the database. If a model has a non-null `deleted_at` value, the model has been soft deleted. To enable soft deletes for a model, use the `Illuminate\Database\Eloquent\SoftDeletes` trait on the model and add the `deleted_at` column to your `$dates` property:

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Flight extends Model
{
    use SoftDeletes;
}
```

```

    /**
     * The attributes that should be mutated to dates.
     *
     * @var array
     */
    protected $dates = ['deleted_at'];
}

```

Of course, you should add the `deleted_at` column to your database table. The [Laravel schema builder](#) contains a helper method to create this column:

```

Schema::table('flights', function ($table)
{
    $table->softDeletes();
});

```

Now, when you call the `delete` method on the model, the `deleted_at` column will be set to the current date and time. And, when querying a model that uses soft deletes, the soft deleted models will automatically be excluded from all query results.

To determine if a given model instance has been soft deleted, use the `trashed` method:

```

if ($flight->trashed()) {
    //
}

```

Querying Soft Deleted Models

Including Soft Deleted Models

As noted above, soft deleted models will automatically be excluded from query results. However, you may force soft deleted models to appear in a result set using the `withTrashed` method on the query:

```

$flights = App\Flight::withTrashed() -
    >where('account_id', 1)
    ->get();

```

The `withTrashed` method may also be used on a [relationship](#) query:

```

$flight->history()->withTrashed()->get();

```

Where Clause Caveats

When adding `orWhere` clauses to your queries on soft deleted models, always use [advance where clauses](#) to logically group the `WHERE` clauses. For example:

```

User::where(function($query) {
    $query->where('name', '=', 'John')
        ->orWhere('votes', '>', 100);
}) -
    >get();

```

This will produce the following SQL:

```
select * from `users` where `users`.`deleted_at` is null and (`name` = 'John' or `votes` > 100)
```

If the `orWhere` clause is not grouped, it will produce the following SQL which will contain soft deleted records:

```
select * from `users` where `users`.`deleted_at` is null and `name` = 'John' or `votes` > 100
```

Retrieving Only Soft Deleted Models

The `onlyTrashed` method will retrieve **only** soft deleted models:

```
$flights = App\Flight::onlyTrashed() -
    >where('airline_id', 1)
    ->get();
```

Restoring Soft Deleted Models

Sometimes you may wish to "un-delete" a soft deleted model. To restore a soft deleted model into an active state, use the `restore` method on a model instance:

```
$flight->restore();
```

You may also use the `restore` method in a query to quickly restore multiple models:

```
App\Flight::withTrashed() -
    >where('airline_id', 1)
    ->restore();
```

Like the `withTrashed` method, the `restore` method may also be used on relationships:

```
$flight->history()->restore();
```

Permanently Deleting Models

Sometimes you may need to truly remove a model from your database. To permanently remove a soft deleted model from the database, use the `forceDelete` method:

```
// Force deleting a single model instance...
$flight->forceDelete();
```

```
// Force deleting all related models...
$flight->history()->forceDelete();
```

Chapter – 5.2 [Validation]

☺ Defining the Routes:

First, let's assume we have the following routes defined in our `app/Http/routes.php` file:

```
// Display a form to create a blog post...
Route::get('post/create', 'PostController@create');

// Store a new blog post...
Route::post('post', 'PostController@store');
```

Of course, the `GET` route will display a form for the user to create a new blog post, while the `POST` route will store the new blog post in the database.

☺ Creating the controller:

Next, let's take a look at a simple controller that handles these routes. We'll leave the `store` method empty for now:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Http\Controllers\Controller;

class PostController extends Controller
{
    /**
     * Show the form to create a new blog post.
     *
     * @return Response
     */
    public function create()
    {
        return view('post.create');
    }

    /**
     * Store a new blog post.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        // Validate and store the blog post...
    }
}
```


☺ Writing the validation Logic:

Now we are ready to fill in our `store` method with the logic to validate the new blog post. If you examine your application's base controller (`App\Http\Controllers\Controller`) class, you will see that the class uses a `ValidatesRequests` trait. This trait provides a convenient `validate` method in all of your controllers.

The `validate` method accepts an incoming HTTP request and a set of validation rules. If the validation rules pass, your code will keep executing normally; however, if validation fails, an exception will be thrown and the proper error response will automatically be sent back to the user. In the case of a traditional HTTP request, a redirect response will be generated, while a JSON response will be sent for AJAX requests.

To get a better understanding of the `validate` method, let's jump back into the `store` method:

```
/**
 * Store a new blog post.
 *
 * @param Request $request
 * @return Response
 */
public function store(Request $request)
{
    $this->validate($request, [
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ]);

    // The blog post is valid, store in database...
}
```

As you can see, we simply pass the incoming HTTP request and desired validation rules into the `validate` method. Again, if the validation fails, the proper response will automatically be generated. If the validation passes, our controller will continue executing normally.

A Note On Nested Attributes

If your HTTP request contains "nested" parameters, you may specify them in your validation rules using "dot" syntax:

```
$this->validate($request, [
    'title' => 'required|unique:posts|max:255',
    'author.name' => 'required',
    'author.description' => 'required',
]);
```

☺ Displaying the validation errors:

So, what if the incoming request parameters do not pass the given validation rules? As mentioned previously, Laravel will automatically redirect the user back to their previous location. In addition, all of the validation errors will automatically be flushed to the session.

Again, notice that we did not have to explicitly bind the error messages to the view in our GET route. This is because Laravel will always check for errors in the session data, and automatically bind them to the view if they are available. **So, it is important to note that an `$errors` variable will always be available in all of your views on every request**, allowing you to conveniently assume the `$errors` variable is always defined and can be safely used. The `$errors` variable will be an instance of `Illuminate\Support\MessageBag`. For more information on working with this object, check out its documentation.

So, in our example, the user will be redirected to our controller's `create` method when validation fails, allowing us to display the error messages in the view:

```
<!-- /resources/views/post/create.blade.php -->

<h1>Create Post</h1>

@if (count($errors) > 0)
    <div class="alert alert-danger">
        <ul>
            @foreach ($errors->all() as
                $error) <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

<!-- Create Post Form -->
```

Customizing The Flashed Error Format

If you wish to customize the format of the validation errors that are flashed to the session when validation fails, override the `formatValidationErrors` on your base controller. Don't

forget to import the `Illuminate\Contracts\Validation\Validator` class at the top of the

file:

```
<?php
```

```
namespace
```

```
App\Http\Controllers;
```

```
use Illuminate\Foundation\Bus\DispatchesJobs;
use Illuminate\Contracts\Validation\Validator;
use Illuminate\Routing\Controller as BaseController; use
Illuminate\Foundation\Validation\ValidatesRequests;
```

```
abstract class Controller extends BaseController
{
```

```

use DispatchesJobs, ValidatesRequests;

/**
 * {@inheritdoc}
 */
protected function formatValidationErrors(Validator $validator)
{
    return $validator->errors()->all();
}
}

```

☺ Array Validations

```

$validator = Validator::make($request->all(), [
    'name.*' => 'required|distinct|min:3',
    'amount.*' => 'required|integer|min:1',
    'description.*' => 'required|string'
]);

$this->validate($request, [
    'title' => 'required|unique:posts|max:255',
    'body' => 'required',
    'publish_at' => 'nullable|date',
]);

```

As you can see, we simply pass the incoming HTTP request and desired validation rules into the validate method. Again, if the validation fails, the proper response will automatically be generated. If the validation passes, our controller will continue executing normally. Laravel will check for errors in the session data, and automatically bind them to the view if they are available. When check the Multiple Elements than make in Array Stopping

☺ Creating new validators:

MANUALLY CREATING VALIDATORS

If you do not want to use the `ValidatesRequests` trait's `validate` method, you may create a validator instance manually using the `Validator` facade. The `make` method on the facade generates a new validator instance:

```

<?php

namespace App\Http\Controllers;

use Validator;
use Illuminate\Http\Request;
use App\Http\Controllers\Controller;

class PostController extends Controller
{
    /**
     * Store a new blog post.
     *
     * @param Request $request
     * @return Response
     */
    public function store(Request $request)
    {
        $validator = Validator::make($request->all(), [
            'title' => 'required|unique:posts|max:255',
            'body' => 'required',

```

```

    });

    if ($validator->fails()) {
        return redirect('post/create') -
            >withErrors($validator)
            ->withInput();
    }

    // Store the blog post...
}
}

```

The first argument passed to the `make` method is the data under validation. The second argument is the validation rules that should be applied to the data.

After checking if the request failed to pass validation, you may use the `withErrors` method to flash the error messages to the session. When using this method, the `$errors` variable will automatically be shared with your views after redirection, allowing you to easily display them back to the user. The `withErrors` method accepts a validator, a `MessageBag`, or a PHP array.

Named Error Bags

If you have multiple forms on a single page, you may wish to name the `MessageBag` of errors, allowing you to retrieve the error messages for a specific form. Simply pass a name as the second argument to `withErrors`:

```

return redirect('register') -
    >withErrors($validator, 'login');

```

You may then access the named `MessageBag` instance from the `$errors` variable:

```

{{ $errors->login->first('email') }}

```

After Validation Hook

The validator also allows you to attach callbacks to be run after validation is completed. This allows you to easily perform further validation and even add more error messages to the message collection. To get started, use the `after` method on a validator instance:

```

$validator = Validator::make(...);

$validator->after(function($validator) {
    if ($this->somethingElseIsInvalid()) {
        $validator->errors()->add('field', 'Something is wrong
with this field!');
    }
});

if ($validator->fails()) {
    //
}

```

☺ Error messages and custom errors:

After calling the `errors` method on a `Validator` instance, you will receive an `Illuminate\Support\MessageBag` instance, which has a variety of convenient methods for working with error messages.

Retrieving The First Error Message For A Field

To retrieve the first error message for a given field, use the `first` method:

```
$messages = $validator->errors();  
  
echo $messages->first('email');
```

Retrieving All Error Messages For A Field

If you wish to simply retrieve an array of all of the messages for a given field, use the `get` method:

```
foreach ($messages->get('email') as $message) {  
    //  
}
```

Retrieving All Error Messages For All Fields

To retrieve an array of all messages for all fields, use the `all` method:

```
foreach ($messages->all() as $message) {  
    //  
}
```

Determining If Messages Exist For A Field

```
if ($messages->has('email'))  
{  
    //  
}
```

Retrieving An Error Message With A Format

```
echo $messages->first('email', '<p>:message</p>');
```

Retrieving All Error Messages With A Format

```
foreach ($messages->all('<li>:message</li>') as $message) {  
    //  
}
```

CUSTOM ERROR MESSAGES

If needed, you may use custom error messages for validation instead of the defaults. There are several ways to specify custom messages. First, you may pass the custom messages as the third argument to the `Validator::make` method:

```
$messages = [
    'required' => 'The :attribute field is required.',
];

$validator = Validator::make($input, $rules, $messages);
```

In this example, the `:attribute` place-holder will be replaced by the actual name of the field under validation. You may also utilize other place-holders in validation messages. For example:

```
$messages = [
    'same'      => 'The :attribute and :other must match.',
    'size'      => 'The :attribute must be exactly :size.',
    'between'   => 'The :attribute must be between :min - :max.',
    'in'        => 'The :attribute must be one of the following types:
:values',
];
```

Specifying A Custom Message For A Given Attribute

Sometimes you may wish to specify a custom error messages only for a specific field. You may do so using "dot" notation. Specify the attribute's name first, followed by the rule:

```
$messages = [
    'email.required' => 'We need to know your e-mail address!',
];
```

Specifying Custom Messages In Language Files

In many cases, you may wish to specify your attribute specific custom messages in a language file instead of passing them directly to the `Validator`. To do so, add your messages to custom array in the `resources/lang/xx/validation.php` language file.

```
'custom' => [
    'email' => [
        'required' => 'We need to know your e-mail address!',
    ],
],
```

☺ Available Validators:

1. *accepted*

The field under validation must be *yes*, *on*, *1*, or *true*. This is useful for validating "Terms of Service" acceptance.

2. *after:date*

The field under validation must be a value after a given date. The dates will be passed into the `strtotime` PHP function:

```
'start_date' => 'required|date|after:tomorrow'
```

Instead of passing a date string to be evaluated by `strtotime`, you may specify another field to compare against the date:

```
'finish_date' => 'required|date|after:start_date'
```

3. *alpha*

The field under validation must be entirely alphabetic characters.

4. *alpha_dash*

The field under validation may have alpha-numeric characters, as well as dashes and underscores.

5. *alpha_num[alpha_numeric]*

The field under validation must be entirely alpha-numeric characters.

6. *array*

The field under validation must be a PHP `array`.

7. *before:date*

The field under validation must be a value preceding the given date. The dates will be passed into the PHP `strtotime` function.

8. *between:min,max*

The field under validation must have a size between the given *min* and *max*. Strings, numerics, and files are evaluated in the same fashion as the size rule.

9.boolean

The field under validation must be able to be cast as a boolean. Accepted input are `true`, `false`, `1`, `0`, `"1"`, and `"0"`.

10.date

The field under validation must be a valid date according to the `strtotime` PHP function.

11.date_format:format

The field under validation must match the given *format*. The format will be evaluated using the PHP `date_parse_from_format` function. You should use **either** `date` or `date_format` when validating a field, not both.

12.different:field

The field under validation must have a different value than *field*.

13.digits:value

The field under validation must be *numeric* and must have an exact length of *value*.

14.digits_between:min,max

The field under validation must have a length between the given *min* and *max*.

15.email

The field under validation must be formatted as an e-mail address.

16.exists:table,column

The field under validation must exist on a given database table.

17.image

The file under validation must be an image (jpeg, png, bmp, gif, or svg)

18.in:foo,bar,...

The field under validation must be included in the given list of values.

19.integer

The field under validation must be an integer.

20.max:value

The field under validation must be less than or equal to a maximum *value*. Strings, numerics, and files are evaluated in the same fashion as the size rule.

21.min:value

The field under validation must have a minimum *value*. Strings, numerics, and files are evaluated in the same fashion as the size rule.

22.not_in:foo,bar,...

The field under validation must not be included in the given list of values.

23.numeric

The field under validation must be numeric.

24.regex:pattern [Regular Expression]

The field under validation must match the given regular expression.

Note: When using the `regex` pattern, it may be necessary to specify rules in an array instead of using pipe delimiters, especially if the regular expression contains a pipe character.

25.required

The field under validation must be present in the input data and not empty. A field is considered "empty" if one of the following conditions are true:

- The value is `null`.
- The value is an empty string.
- The value is an empty array or empty `Countable` object.
- The value is an uploaded file with no path.

26.string

The field under validation must be a string.

😊 Custom validation rules:

Laravel provides a variety of helpful validation rules; however, you may wish to specify some of your own. One method of registering custom validation rules is using the `extend` method on the `Validator` facade. Let's use this method within a service provider to register a custom validation rule:

```
<?php

namespace App\Providers;

use Validator;
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Bootstrap any application services.
     *
     * @return void */
    public function boot()
    {
        Validator::extend('foo', function($attribute, $value,
        $parameters, $validator) {
            return $value == 'foo';
        });
    }

    /**
     * Register the service provider.
     *
     * @return void */
    public function register()
    {
        //
    }
}
```

The custom validator Closure receives four arguments: the name of the `$attribute` being validated, the `$value` of the attribute, an array of `$parameters` passed to the rule, and the `Validator` instance.

You may also pass a class and method to the `extend` method instead of a Closure:

```
Validator::extend('foo', 'FooValidator@validate');
```

Defining The Error Message

You will also need to define an error message for your custom rule. You can do so either using an inline custom message array or by adding an entry in the validation language file. This message should be placed in the first level of the array, not within the `custom` array, which is only for attribute-specific error messages:

```
"foo" => "Your input was invalid!",

"accepted" => "The :attribute must be accepted.",

// The rest of the validation error messages...
```

When creating a custom validation rule, you may sometimes need to define custom placeholder replacements for error messages. You may do so by creating a custom Validator as described above then making a call to the `replacer` method on the `Validator` facade. You may do this within the `boot` method of a service provider:

```
/**
 * Bootstrap any application services.
 */
* @return void
*/
public function boot()
{
    Validator::extend(...);

    Validator::replacer('foo', function($message, $attribute, $rule,
    $parameters) {
        return str_replace(...);
    });
}
```

Implicit Extensions

By default, when an attribute being validated is not present or contains an empty value as defined by the `required` rule, normal validation rules, including custom extensions, are not run. For example, the integer rule will not be run against a `null` value:

```
$rules = ['count' => 'integer'];

$input = ['count' => null];

Validator::make($input, $rules)->passes(); // true
```

For a rule to run even when an attribute is empty, the rule must imply that the attribute is required. To create such an "implicit" extension, use the `Validator::extendImplicit()` method:

```
Validator::extendImplicit('foo', function($attribute, $value,
    $parameters, $validator) {
    return $value == 'foo';
});
```

```
});
```

Note: An "implicit" extension only *implies* that the attribute is required. Whether it actually invalidates a missing or empty attribute is up to you.

❖ Database related programs [mysql connectivity] –

➤ Database and table same in all programs

1->Create New Database Name STUDENT1 in mysql database

2->Go to Laravel Project and Open Env File

```
===>DB_DATABASE=STUDENT(nameofdatabase)
```

```
===>DB_USERNAME=root
```

```
===>DB_PASSWORD="",
```

3->Connection is created Succssfully

```
=====CreateTables=in Database=====
```

4->we Create new table in Database with command

```
==>open CMD
```

```
=====>php artisan make:model student -m
```

now Table is Created....

-go to laravel Project In Database Folder In Migration Folder

```
---->open the 2017_08_000212_create_student_table.php
```

change this file in UP() FUNCTION

```
<?php
```

```
use Illuminate\Database\Schema\Blueprint;
```

```
use Illuminate\Database\Migrations\Migration;
```

```
class CreateStudentsTable extends Migration
```

```
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('students', function (Blueprint $table) {
            $table->increments('id');
            $table->string('name');
            $table->string('address');
            $table->string('Mo');
            $table->string('Gender');
            $table->timestamps();
        });
    }
}
```

```

/**
 * Reverse the migrations.
 *
 * @return void
 */
public function down()
{
    Schema::drop('students');
}
}
?>

```

==>Now Goto Cmd and write this command

php artisan migrate

-----ok NOW Table is Created in Database check.

Write following command in command prompt

Php artisan make:auth

Database folder -> migration folder -> two files create

Usertable.php

Passwordreset.php

1. Write a laravel program for student detail to perform following operations on student data using SQL Query.

Operations: insert, delete, update, select

1)Routes.php

<?php

```

/*
|-----
| Application Routes
|-----
|
| Here is where you can register all of the routes for an application.
| It's a breeze. Simply tell Laravel the URIs it should respond to
| and give it the controller to call when that URI is requested.
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('/AddStudent','StudentController@AddStudent');
Route::post('/PostStudent','StudentController@PostStudent');

Route::get('/SelectStudent','StudentController@ShowStudent');
Route::get('/updateStudent/{id}','StudentController@updateStudent');
Route::post('/updateStudent/{id}','StudentController@postupdateStudent');

Route::get('/deleteStd/{id}','StudentController@DeleteStudent');

Route::auth();

Route::get('/home', 'HomeController@index');

```

2)Controlller.php

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

use App\Http\Requests;
use DB;
use App\student;

class StudentController extends Controller
{

    public function __construct()
    {
        $this->middleware('auth');
    }
    public function AddStudent()
    {

        return view('RegisterFrom');

    }
    public function PostStudent(Request $res)
    {

        //RAW SQL Queries:
        DB::insert('insert into students (name,address,MobilenNo,email) values ( ?,
            ?,?,?)',
            [$res->input('name'),
            $res->input('Address'),
            $res->input('mobile'),
            $res->input('Email')
            ]);

        return redirect('/SelectStudent');
    }
    public function ShowStudent()
    {
        //RAW SQL Queries
        //$std = DB::select('select * from students');

        //Eloquent ORM Models
        $std =student::all();

        return view('selectStudent',compact('std'));
    }
    public function updateStudent($id)
    {
        //RAW SQL Queries

        //$std = DB::select('select * from students where id = ?',[$id]);

        //Eloquent ORM Models

        $std=student::find($id);

        return view('update',compact('std'));
    }
}

```

```

    public function postupdateStudent(Request $res,$id)
    {
        //RAW SQL Queries
        DB::table('students')
            ->where('id', $id)
            ->update(
                ['name'=>$res->input('name'),
                'address'=>$res->input('Address'),
                'MobileNo'=>$res->input('mobile'),
                'email'=>$res->input('Email')
                ]);

        return redirect('/SelectStudent');
    }
    public function DeleteStudent($id)
    {
        //RAW SQL Queries
        $deleted = DB::delete('delete from students where id=?',[$id]);

        return redirect('/SelectStudent');
    }
}
?>

```

3) RegisterForm.blade.php

```

<!DOCTYPE html>
<html>
<head>
    <title></title>
<link rel="stylesheet" href="css/bootstrap.min.css">

<!-- jQuery library -->
<script src="jquery-3.2.1.min.js"></script>

<!-- Latest compiled JavaScript -->
<script src="js/bootstrap.min.js"></script>
</head>
<body>

</body>
</html>
<div class="container">
<h1 class="well">Registration Form <br><center><a href="{{url('/AddStudent')}}" class="btn btn-
primary">AddStudent</a>| <a href="{{url('/SelectStudent')}}" class="btn btn-
primary">ShowStudent</a>| </center></h1>
    <div class="col-lg-12 well">
        <div class="row">
            <form action="/PostStudent" method="post">

                <input type="hidden" name="_token" value="{{ csrf_token() }}">
                <div class="col-sm-12">
                    <div class="row">
                        <div class="col-sm-6 form-group">
                            <label>First Name</label>
                            <input type="text" placeholder="Enter
First Name Here.." class="form-control" name="name">
                        </div>

                        </div>
                        <div class="form-group">
                            <label>Address</label>
                            <textarea placeholder="Enter Address Here.."
rows="3" class="form-control" name="Address"></textarea>

```

```

        </div>

        <div class="form-group">
            <label>Phone Number</label>
            <input type="text" placeholder="Enter Phone Number
Here.." class="form-control" name="mobile">
        </div>
        <div class="form-group">
            <label>Email Address</label>
            <input type="text" placeholder="Enter Email Address
Here.." class="form-control" name="Email">
        </div>

        <input type="Submit" class="btn btn-lg btn-info"
value="Submit">
    </div>
</form>
</div>
</div>
</div>

```

4) selectStudent.blade.php

```

<!DOCTYPE html>
<html lang="en">
<head>
<title>Bootstrap Example</title>
<link rel="stylesheet" href="css/bootstrap.min.css">
<!-- jQuery library -->
<script src="jquery-3.2.1.min.js"></script>
<!-- Latest compiled JavaScript -->
<script src="js/bootstrap.min.js"></script>
</head>
<body>

<div class="container">
<h1 class="well">Student Data
<br><center><a href="{{url('/AddStudent')}}" class="btn btn-primary">AddStudent</a>|<a
href="{{url('/SelectStudent')}}" class="btn btn-primary">ShowStudent</a>|</center></h1>
<h2>Student List</h2>

<table class="table">
<thead>
<tr>
<th>Id</th>
<th>Name</th>
<th>Address</th>
<th>Gender</th>
<th>Action</th>
</tr>
</thead>
<tbody>
<tr>
<td>
@foreach($std as $std)
<td>{{ $std->name }}</td>
<td>{{ $std->address }}</td>
<td>{{ $std->MobileNo }}</td>
<td>{{ $std->email }}</td>
<td>
<a href="{{url('/updateStudent')}}/{{ $std->id }}" class="btn btn-primary">UpdateStudent</a>|<a
href="{{url('/deleteStd')}}/{{ $std->id }}" class="btn btn-primary">DeleteStudent</a>
</td>
</tr>

```



```

        @endforeach
    </tbody>
</table>
</div>

</body>
</html>

```

5) Update.blade.php

```

<!DOCTYPE html>
<html>
<head>
    <title></title>
    <link rel="stylesheet" href="css/bootstrap.min.css">

    <!-- jQuery library -->
    <script src="jquery-3.2.1.min.js"></script>

    <!-- Latest compiled JavaScript -->
    <script src="js/bootstrap.min.js"></script>
</head>
<body>

</body>
</html>
<div class="container">
<h1 class="well">UpdateStudent Form <br><center><a href="{{url('/AddStudent')}}" class="btn btn-
primary">AddStudent</a>| <a href="{{url('/SelectStudent')}}" class="btn btn-
primary">ShowStudent</a>| </center></h1>
    <div class="col-lg-12 well">
        <div class="row">
            <form action="/updateStudent/{{ $std->id }}" method="post">
                <input type="hidden" name="_token" value="{{ csrf_token() }}">
                <div class="col-sm-12">
                    <div class="row">
                        <div class="col-sm-6 form-group">
                            <label>First Name</label>
                            <input type="text" placeholder="Enter
First Name Here.." class="form-control" value="{{ $std->name }}" name="name">
                        </div>
                        <div class="col-sm-6 form-group">
                            <label>Address</label>
                            <textarea placeholder="Enter Address Here.."
rows="3" class="form-control" name="Address">{{ $std->address }}</textarea>
                        </div>
                    </div>
                    <div class="form-group">
                        <label>Phone Number</label>
                        <input type="text" placeholder="Enter Phone Number
Here.." class="form-control" value="{{ $std->MobileNo }}" name="mobile">
                    </div>
                    <div class="form-group">
                        <label>Email Address</label>
                        <input type="text" placeholder="Enter Email Address
Here.." class="form-control" value="{{ $std->email }}" name="Email">
                    </div>
                    <input type="Submit" class="btn btn-lg btn-info" value="Submit">
                </div>
            </form>
        </div>
    </div>
</div>

```

```

        </form>
    </div>
</div>

```

2. Write a laravel program for student detail to perform following operations on student data using model. [Connectivity with Model]

Operations: insert, delete, update, select

1) Routes.php

```

<?php

/*
|-----
| Application Routes
|-----
|
| Here is where you can register all of the routes for an application.
| It's a breeze. Simply tell Laravel the URIs it should respond to
| and give it the controller to call when that URI is requested.
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('/AddStudent', "StudentController@AddStudent");
Route::post('/PostStudent', "StudentController@PostStudent");

Route::get('/SelectStudent', "StudentController@ShowStudent");
Route::get('/updateStudent/{id}', "StudentController@updateStudent");
Route::post('/updateStudent/{id}', "StudentController@postupdateStudent");

Route::get('/deleteStd/{id}', "StudentController@DeleteStudent");

Route::auth();

Route::get('/home', 'HomeController@index');

```

2)student.php (in model folder)

```

<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class student extends Model
{
    //
    protected $table = 'students';
    protected $dates = ['deleted_at'];
}

```

3)StudentController .php

```

<?php

namespace App\Http\Controllers;

```

```

use Illuminate\Http\Request;

use App\Http\Requests;
use DB;
use App\student;

class StudentController extends Controller
{

    public function __construct()
    {
        $this->middleware('auth');
    }
    public function AddStudent()
    {

        return view('RegisterFrom');

    }
    public function PostStudent(Request $res)
    {

        //Eloquent ORM Models

        $std= new student;
        $std->name=$res->input('name');
        $std->address=$res->get('Address');
        $std->MobileNo=$res->get('mobile');
        $std->email=$res->input('Email');
        $std->save();

        return redirect('/SelectStudent');
    }
    public function ShowStudent()
    {

        //Eloquent ORM Models
        $std =student::all();

        return view('selectStudent',compact('std'));

    }
    public function updateStudent($id)
    //Eloquent ORM Models

        $std=student::find($id);

        return view('update',compact('std'));
    }
    public function postupdateStudent(Request $res,$id)
    {
        //Eloquent ORM Models

        $std= student::find($id);
        $std->name=$res->input('name');
        $std->address=$res->get('Address');
        $std->MobileNo=$res->get('mobile');
        $std->email=$res->input('Email');
        $std->save();

        return redirect('/SelectStudent');
    }

```

```

    }
    public function DeleteStudent($id)
    {

        //Eloquent ORM Models
        $std=student::find($id);
        $std->delete();
        return redirect('/SelectStudent');
    }
}
?>

```

4) RegisterForm.blade.php

```

<!DOCTYPE html>
<html>
<head>
    <title></title>
<link rel="stylesheet" href="css/bootstrap.min.css">

<!-- jQuery library -->
<script src="jquery-3.2.1.min.js"></script>

<!-- Latest compiled JavaScript -->
<script src="js/bootstrap.min.js"></script>
</head>
<body>

</body>
</html>
<div class="container">
<h1 class="well">Registration Form <br><center><a href="{{url('/AddStudent')}}" class="btn btn-
primary">AddStudent</a>| <a href="{{url('/SelectStudent')}}" class="btn btn-
primary">ShowStudent</a>| </center></h1>
    <div class="col-lg-12 well">
        <div class="row">
            <form action="/PostStudent" method="post">

                <input type="hidden" name="_token" value="{{ csrf_token() }}">
                <div class="col-sm-12">
                    <div class="row">
                        <div class="col-sm-6 form-group">
                            <label>First Name</label>
                            <input type="text" placeholder="Enter
First Name Here.." class="form-control" name="name">
                        </div>
                        <div class="col-sm-6 form-group">
                            <label>Address</label>
                            <textarea placeholder="Enter Address Here.."
rows="3" class="form-control" name="Address"></textarea>
                        </div>
                    </div>
                    <div class="form-group">
                        <label>Phone Number</label>
                        <input type="text" placeholder="Enter Phone Number
Here.." class="form-control" name="mobile">
                    </div>
                    <div class="form-group">
                        <label>Email Address</label>

```

```

        <input type="text" placeholder="Enter Email Address
Here.." class="form-control" name="Email">
    </div>

    <input type="Submit" class="btn btn-lg btn-info"
value="Submit">
    </div>
</form>
</div>
</div>
</div>
</div>

```

5) selectStudent.blade.php

```

<!DOCTYPE html>
<html lang="en">
<head>
<title>Bootstrap Example</title>
<link rel="stylesheet" href="css/bootstrap.min.css">

<!-- jQuery library -->
<script src="jquery-3.2.1.min.js"></script>

<!-- Latest compiled JavaScript -->
<script src="js/bootstrap.min.js"></script>
</head>
<body>

<div class="container">
<h1 class="well">Student Data
<br><center><a href="{{url('/AddStudent')}}" class="btn btn-primary">AddStudent</a>|<a
href="{{url('/SelectStudent')}}" class="btn btn-primary">ShowStudent</a>|</center></h1>
<h2>Student List</h2>

<table class="table">
<thead>
<tr>
<th>Id</th>
<th>Name</th>
<th>Address</th>
<th>Gender</th>
<th>Action</th>
</tr>
</thead>
<tbody>
    <tr>
        @foreach($std as $std)
        <td>{{ $std->name }}</td>
        <td>{{ $std->address }}</td>
        <td>{{ $std->MobileNo }}</td>
        <td>{{ $std->email }}</td>
        <td>
            <a href="{{url('/updateStudent')}}/{{ $std->id }}" class="btn btn-primary">UpdateStudent</a>|<a
href="{{url('/deleteStd')}}/{{ $std->id }}" class="btn btn-primary">DeleteStudent</a>
        </td>
        </tr>
    @endforeach
</tbody>
</table>
</div>

</body>
</html>

```

6) Update.blade.php

```

<!DOCTYPE html>
<html>
<head>
    <title></title>
<link rel="stylesheet" href="css/bootstrap.min.css">

<!-- jQuery library -->
<script src="jquery-3.2.1.min.js"></script>

<!-- Latest compiled JavaScript -->
<script src="js/bootstrap.min.js"></script>
</head>
<body>

</body>
</html>
<div class="container">
<h1 class="well">UpdateStudent Form <br><center><a href="{{url('/AddStudent')}}" class="btn btn-
primary">AddStudent</a>|<a href="{{url('/SelectStudent')}}" class="btn btn-
primary">ShowStudent</a>|</center></h1>
    <div class="col-lg-12 well">
        <div class="row">
            <form action="/updateStudent/{{${std->id}}" method="post">
                <input type="hidden" name="_token" value="{{csrf_token()}}">
                <div class="col-sm-12">
                    <div class="row">
                        <div class="col-sm-6 form-group">
                            <label>First Name</label>
                            <input type="text" placeholder="Enter
First Name Here.." class="form-control" value="{{${std->name}}" name="name">
                        </div>
                        <div class="col-sm-6 form-group">
                            <label>Address</label>
                            <textarea placeholder="Enter Address Here.."
rows="3" class="form-control" name="Address">{{${std->address}}</textarea>
                        </div>
                    </div>
                    <div class="form-group">
                        <label>Phone Number</label>
                        <input type="text" placeholder="Enter Phone Number
Here.." class="form-control" value="{{${std->MobileNo}}" name="mobile">
                    </div>
                    <div class="form-group">
                        <label>Email Address</label>
                        <input type="text" placeholder="Enter Email Address
Here.." class="form-control" value="{{${std->email}}" name="Email">
                    </div>
                    <input type="Submit" class="btn btn-lg btn-info" value="Submit">
                </div>
            </form>
        </div>
    </div>
</div>

```

3. Write a laravel program for student detail to perform following operations on student data using SQL Query with validation.

Operations: insert, delete, update, select

1) Routes.php

```
<?php

/*
|-----
| Application Routes
|-----
|
| Here is where you can register all of the routes for an application.
| It's a breeze. Simply tell Laravel the URIs it should respond to
| and give it the controller to call when that URI is requested.
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('/AddStudent','StudentController@AddStudent');
Route::post('/PostStudent','StudentController@PostStudent');

Route::get('/SelectStudent','StudentController@ShowStudent');
Route::get('/updateStudent/{id}','StudentController@updateStudent');
Route::post('/updateStudent/{id}','StudentController@postupdateStudent');

Route::get('/deleteStd/{id}','StudentController@DeleteStudent');

Route::auth();

Route::get('/home', 'HomeController@index');
```

2)studentController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

use App\Http\Requests;
use DB;
use App\student;

class StudentController extends Controller
{

    public function __construct()
    {
        $this->middleware('auth');
    }
    public function AddStudent()
    {

        return view('RegisterFrom');
```

```

    }
    public function PostStudent(Request $res)
    {

        //RAW SQL Queries:
        DB::insert('insert into students (name,address,MobilenNo,email) values ( ?,
        ?,?,?)',
        [$res->input('name'),
        $res->input('Address'),
        $res->input('mobile'),
        $res->input('Email')
        ]);

        return redirect('/SelectStudent');
    }
    public function ShowStudent()
    {
        //RAW SQL Queries
        $std = DB::select('select * from students');

        return view('selectStudent',compact('std'));
    }
    public function updateStudent($id)
    {
        //RAW SQL Queries

        $std = DB::select('select * from students where id = ?',[$id]);

        return view('update',compact('std'));
    }
    public function postupdateStudent(Request $res,$id)
    {
        //RAW SQL Queries
        DB::table('students')
        ->where('id', $id)
        ->update(
        ['name'=>$res->input('name'),
        'address'=>$res->input('Address'),
        'MobileNo'=>$res->input('mobile'),
        'email'=>$res->input('Email')
        ]);

        return redirect('/SelectStudent');
    }
    public function DeleteStudent($id)
    {
        //RAW SQL Queries
        $deleted = DB::delete('delete from students where id=?',[$id]);

        return redirect('/SelectStudent');
    }
}
?>

```

3)RegisterFrom.blade.php

```

<!DOCTYPE html>
<html>
<head>

```



```

</title></title>
<link rel="stylesheet" href="css/bootstrap.min.css">

<!-- jQuery library -->
<script src="jquery-3.2.1.min.js"></script>

<!-- Latest compiled JavaScript -->
<script src="js/bootstrap.min.js"></script>
</head>
<body>

</body>
</html>
<div class="container">
<h1 class="well">Registration Form <br><center><a href="{{url('/AddStudent')}}" class="btn btn-
primary">AddStudent</a>| <a href="{{url('/SelectStudent')}}" class="btn btn-
primary">ShowStudent</a>| </center></h1>
    <div class="col-lg-12 well">
        <div class="row">
            @if(count($errors)>0)

                <div class="alert alert-danger">
                    <ul>
                        @foreach($errors->all() as $error )
                            <li>{{$error}}</li>
                        @endforeach
                    </ul>
                </div>

            @endif

            <form action="/PostStudent" method="post">

                <input type="hidden" name="_token" value="{{ csrf_token() }}">
                <div class="col-sm-12">
                    <div class="row">
                        <div class="col-sm-6 form-group">
                            <label>First Name</label>
                            <input type="text" placeholder="Enter
First Name Here.." class="form-control" name="name">
                            @if ($errors->has('name')) <p
class="help-block">{{ $errors->first('name') }}</p> @endif
                        </div>

                        <div class="col-sm-6 form-group">
                            <label>Address</label>
                            <textarea placeholder="Enter Address Here.."
rows="3" class="form-control" name="Address"></textarea>
                        </div>

                        <div class="form-group">
                            <label>Phone Number</label>
                            <input type="text" placeholder="Enter Phone Number
Here.." class="form-control" name="mobile">
                        </div>

                        <div class="form-group">
                            <label>Email Address</label>
                            <input type="text" placeholder="Enter Email Address
Here.." class="form-control" name="Email">
                            @if ($errors->has('Email')) <p class="alert alert-
danger">{{ $errors->first('Email') }}</p> @endif

```

```

        </div>

        <input type="Submit" class="btn btn-lg btn-info"
value="Submit">

        </div>
    </form>
</div>

</div>
</div>

```

4) selectStudent.blade.php

```

<!DOCTYPE html>
<html lang="en">
<head>
<title>Bootstrap Example</title>
<link rel="stylesheet" href="css/bootstrap.min.css">

<!-- jQuery library -->
<script src="jquery-3.2.1.min.js"></script>

<!-- Latest compiled JavaScript -->
<script src="js/bootstrap.min.js"></script>
</head>
<body>

<div class="container">
<h1 class="well">Student Data
<br><center><a href="{{url('/AddStudent')}}" class="btn btn-primary">AddStudent</a>|<a
href="{{url('/SelectStudent')}}" class="btn btn-primary">ShowStudent</a>|</center></h1>
<h2>Student List</h2>

<table class="table">
<thead>
<tr>
<th>Id</th>
<th>Name</th>
<th>Address</th>
<th>Gender</th>
<th>Action</th>
</tr>
</thead>
<tbody>
        <tr>
            <td>
                @foreach($std as $std)
                <td>{{ $std->name }}</td>
                <td>{{ $std->address }}</td>
                <td>{{ $std->MobileNo }}</td>
                <td>{{ $std->email }}</td>
                <td>
                    <a href="{{url('/updateStudent')}}/{{ $std->id }}" class="btn btn-primary">UpdateStudent</a>|<a
href="{{url('/deleteStd')}}/{{ $std->id }}" class="btn btn-primary">DeleteStudent</a>
                </td>
            </tr>
        @endforeach
    </tbody>
</table>
</div>

</body>
</html>

```

5) Update.blade.php

```

<!DOCTYPE html>

```

```

<html>
<head>
    <title></title>
<link rel="stylesheet" href="css/bootstrap.min.css">

<!-- jQuery library -->
<script src="jquery-3.2.1.min.js"></script>

<!-- Latest compiled JavaScript -->
<script src="js/bootstrap.min.js"></script>
</head>
<body>

</body>
</html>
<div class="container">
<h1 class="well">UpdateStudent Form <br><center><a href="{{url('/AddStudent')}}" class="btn btn-
primary">AddStudent</a>|<a href="{{url('/SelectStudent')}}" class="btn btn-
primary">ShowStudent</a>|</center></h1>
    <div class="col-lg-12 well">
        <div class="row">
            <form action="/updateStudent/{{ $std->id }}" method="post">
                <input type="hidden" name="_token" value="{{ csrf_token() }}">
                <div class="col-sm-12">
                    <div class="row">
                        <div class="col-sm-6 form-group">
                            <label>First Name</label>
                            <input type="text" placeholder="Enter
First Name Here.." class="form-control" value="{{ $std->name }}" name="name">
                        </div>
                        <div class="col-sm-6 form-group">
                            <label>Address</label>
                            <textarea placeholder="Enter Address Here.."
rows="3" class="form-control" name="Address">{{ $std->address }}</textarea>
                        </div>
                    </div>
                    <div class="form-group">
                        <label>Phone Number</label>
                        <input type="text" placeholder="Enter Phone Number
Here.." class="form-control" value="{{ $std->MobileNo }}" name="mobile">
                    </div>
                    <div class="form-group">
                        <label>Email Address</label>
                        <input type="text" placeholder="Enter Email Address
Here.." class="form-control" value="{{ $std->email }}" name="Email">
                    </div>
                    <input type="Submit" class="btn btn-lg btn-info" value="Submit">
                </div>
            </form>
        </div>
    </div>
</div>

```

4. Write a laravel program for student detail to perform following operations on student data using model with validation.

Operations: insert, delete, update, select

1) Routes.php

```
<?php

/*
|-----
| Application Routes
|-----
|
| Here is where you can register all of the routes for an application.
| It's a breeze. Simply tell Laravel the URIs it should respond to
| and give it the controller to call when that URI is requested.
|
*/

Route::get('/', function () {
    return view('welcome');
});

Route::get('/AddStudent', "StudentController@AddStudent");
Route::post('/PostStudent', "StudentController@PostStudent");

Route::get('/SelectStudent', "StudentController@showStudent");
Route::get('/updateStudent/{id}', "StudentController@updateStudent");
Route::post('/updateStudent/{id}', "StudentController@postupdateStudent");

Route::get('/deleteStd/{id}', "StudentController@DeleteStudent");

Route::auth();

Route::get('/home', 'HomeController@index');
```

2) student.php (in model folder)

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class student extends Model
{
    //
    protected $table = 'students';
    protected $dates = ['deleted_at'];
}
```

2) studentController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

use App\Http\Requests;
```

```

use DB;
use App\student;

class StudentController extends Controller
{

    public function __construct()
    {
        $this->middleware('auth');
    }
    public function AddStudent()
    {

        return view('RegisterFrom');

    }
    public function PostStudent(Request $res)
    {

        //Eloquent ORM Models

        $std= new student;
        $std->name=$res->input('name');
        $std->address=$res->get('Address');
        $std->MobileNo=$res->get('mobile');
        $std->email=$res->input('Email');
        $std->save();

        return redirect('/SelectStudent');
    }
    public function ShowStudent()
    {
        //Eloquent ORM Models
        $std =student::all();

        return view('selectStudent',compact('std'));

    }
    public function updateStudent($id)
    {

        //Eloquent ORM Models

        $std=student::find($id);

        return view('update',compact('std'));
    }
    public function postupdateStudent(Request $res,$id)
    {

        //Eloquent ORM Models

        $std= student::find($id);
        $std->name=$res->input('name');
        $std->address=$res->get('Address');
        $std->MobileNo=$res->get('mobile');
        $std->email=$res->input('Email');
        $std->save();

        return redirect('/SelectStudent');
    }
    public function DeleteStudent($id)
    {

```

```

        //Eloquent ORM Models
        $std=student::find($id);
        $std->delete();
        return redirect('/SelectStudent');
    }
}
?>

```

4)RegisterFrom.blade.php

```

<!DOCTYPE html>
<html>
<head>
    <title></title>
    <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css">

    <!-- jQuery library -->
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>

    <!-- Latest compiled JavaScript -->
    <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js"></script>
</head>
<body>

</body>
</html>
<div class="container">
<h1 class="well">Registration Form <br><center><a href="{{url('/AddStudent')}}" class="btn btn-
primary">AddStudent</a>| <a href="{{url('/SelectStudent')}}" class="btn btn-
primary">ShowStudent</a>| </center></h1>
    <div class="col-lg-12 well">
        <div class="row">
            @if(count($errors)>0)

                <div class="alert alert-danger">
                    <ul>
                        @foreach($errors->all() as $error )
                            <li>{{$error}}</li>
                        @endforeach
                    </ul>
                </div>

            @endif

            <form action="/PostStudent" method="post">

                <input type="hidden" name="_token" value="{{ csrf_token() }}">
                <div class="col-sm-12">
                    <div class="row">
                        <div class="col-sm-6 form-group">
                            <label>First Name</label>
                            <input type="text" placeholder="Enter
First Name Here.." class="form-control" name="name">
                            @if ($errors->has('name')) <p
class="help-block">{{ $errors->first('name') }}</p> @endif
                        </div>

                        <div class="form-group">
                            <label>Address</label>
                            <textarea placeholder="Enter Address Here.."
rows="3" class="form-control" name="Address"></textarea>
                        </div>
                    </div>
                </div>
            </form>
        </div>
    </div>

```

```

        <div class="form-group">
            <label>Phone Number</label>
            <input type="text" placeholder="Enter Phone Number
Here.." class="form-control" name="mobile">
        </div>
        <div class="form-group">
            <label>Email Address</label>
            <input type="text" placeholder="Enter Email Address
Here.." class="form-control" name="Email">
            @if ($errors->has('Email')) <p class="alert alert-
danger">{{ $errors->first('Email') }}</p> @endif
        </div>
        <input type="Submit" class="btn btn-lg btn-info"
value="Submit">
    </div>
</form>
</div>
</div>

```

5) selectStudent.blade.php

```

<!DOCTYPE html>
<html lang="en">
<head>
<title>Bootstrap Example</title>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css">
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js"></script>
</head>
<body>

<div class="container">
<h1 class="well">Student Data
<br><center><a href="{{url('/AddStudent')}}" class="btn btn-primary">AddStudent</a>|<a
href="{{url('/SelectStudent')}}" class="btn btn-primary">ShowStudent</a>|</center></h1>
<h2>Student List</h2>

<table class="table">
<thead>
<tr>
<th>Id</th>
<th>Name</th>
<th>Address</th>
<th>Gender</th>
<th>Action</th>
</tr>
</thead>
<tbody>
        <tr>
            @foreach($std as $std)
            <td>{{ $std->name }}</td>
            <td>{{ $std->address }}</td>
            <td>{{ $std->MobileNo }}</td>
            <td>{{ $std->email }}</td>
            <td>
                <a href="{{url('/updateStudent')}}/{{ $std->id }}" class="btn btn-primary">UpdateStudent</a>|<a
href="{{url('/deleteStd')}}/{{ $std->id }}" class="btn btn-primary">DeleteStudent</a>
            </td>
        </tr>
    </tbody>
</table>

```

```

</tr>
    @endforeach
</tbody>
</table>
</div>

```

```

</body>
</html>

```

6) Update.blade.php

```

<!DOCTYPE html>
<html>
<head>
    <title></title>
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css">

<!-- jQuery library -->
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>

<!-- Latest compiled JavaScript -->
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js"></script>
</head>
<body>

</body>
</html>
<div class="container">
<h1 class="well">UpdateStudent Form <br><center><a href="{{url('/AddStudent')}}" class="btn btn-
primary">AddStudent</a>|<a href="{{url('/SelectStudent')}}" class="btn btn-
primary">ShowStudent</a>|</center></h1>
    <div class="col-lg-12 well">
        <div class="row">
            <form action="/updateStudent/{{${std->id}}" method="post">
                <input type="hidden" name="_token" value="{{csrf_token()}}">
                <div class="col-sm-12">
                    <div class="row">
                        <div class="col-sm-6 form-group">
                            <label>First Name</label>
                            <input type="text" placeholder="Enter
First Name Here.." class="form-control" value="{{${std->name}}}" name="name">
                        </div>
                        <div class="col-sm-6 form-group">
                            <label>Address</label>
                            <textarea placeholder="Enter Address Here.."
rows="3" class="form-control" name="Address">{{${std->address}}}</textarea>
                        </div>
                        <div class="form-group">
                            <label>Phone Number</label>
                            <input type="text" placeholder="Enter Phone Number
Here.." class="form-control" value="{{${std->MobileNo}}}" name="mobile">
                        </div>
                        <div class="form-group">
                            <label>Email Address</label>
                            <input type="text" placeholder="Enter Email Address
Here.." class="form-control" value="{{${std->email}}}" name="Email">
                        </div>
                    <input type="Submit" class="btn btn-lg btn-info" value="Submit">
                </div>
            </form>
        </div>
    </div>
</div>

```



```
        </div>
    </form>
</div>
</div>
```

BEST OF LUCK